

Open Data Logement

Pipeline ELT sur données publiques françaises : DVF, DPE et INSEE,
de l'API jusqu'au modèle Power BI

Ce document reprend tout ce qui a été construit : le raisonnement, chaque commande lancée, chaque fichier de code en entier, et l'explication de chaque choix. Il est pensé pour être lu dans l'ordre, comme un cours, puis gardé comme référence.

Élément	Détail
Question du projet	Où l'immobilier est-il cher par rapport aux revenus locaux, et quel rôle joue la performance énergétique ?
Périmètre	Hauts-de-France (Aisne 02, Nord 59, Oise 60, Pas-de-Calais 62, Somme 80), ventes 2021 à 2025
Stack	Python · DuckDB · dbt (dbt-duckdb) · Neon PostgreSQL · Power BI Desktop · Git / GitHub
Dépôt	github.com/YvesRollandBoa/open-data-logement
État au 25 septembre 2026	Pipeline complet et testé (70 tests), données exportées dans Neon, mesures DAX prêtes. Prochaine étape : le rapport Power BI.

Auteur : Yves-Rolland Boa · document généré le 25 septembre 2026

Sommaire

1. Le projet en une vue	4
1.1 La question métier	4
1.2 L'architecture	4
1.3 Les choix structurants, et pourquoi	4
1.4 L'arborescence du dépôt	5
1.5 L'entonnoir des données	5
2. Mettre en place l'environnement	6
2.1 Où tourne quoi	6
2.2 L'environnement virtuel Python	6
2.3 Git et GitHub	7
3. Ingestion : télécharger et charger	8
3.1 config.py : les paramètres partagés	8
3.2 download_dvf.py : les ventes immobilières	9
3.3 download_insee.py : les indicateurs communaux	10
3.4 download_dpe.py : les diagnostics via une API paginée	11
3.5 load_raw.py : charger dans DuckDB	13
4. Transformation avec dbt	16
4.1 Les trois couches	16
4.2 Configuration du projet	16
4.3 Les sources	18
4.4 Staging : typer les sources	18
4.5 Intermediate : les règles métier	21
4.6 Marts analytiques	25
4.7 Seed : un référentiel versionné	28
4.8 Le schéma en étoile pour Power BI	29
4.9 Les tests singuliers	33
4.10 Lancer et lire dbt	34
5. Exporter vers Neon	35
5.1 Le secret de connexion : .env	35
5.2 export_neon.py	35
6. Journal des incidents	38
6.1 Un point encore ouvert : les DPE de 2025	38
7. Préparer Power BI	40

7.1 Se connecter à Neon	40
7.2 Monter le modèle	40
7.3 Les mesures DAX	40
7.4 Le rapport visé	45
8. Aide-mémoire	46
8.1 Relancer tout le pipeline	46
8.2 Glossaire	46

1. Le projet en une vue

Le projet reprend l'idée n°12 du dépôt *pimp-your-portfolio* de Gaël Penessot, « ETL Open Data Gouvernement » : construire un entrepôt de données unifié à partir de sources publiques éparpillées. On l'a adapté à ta stack d'Analytics Engineer (dbt, SQL, Power BI) et à un thème concret : le logement.

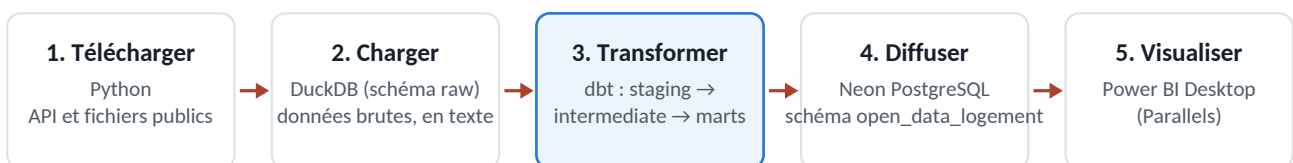
1.1 La question métier

Un projet de portfolio convainc quand il répond à une vraie question. Celle-ci croise trois sources qui, seules, ne disent pas grand-chose :

Source	Producteur	Ce qu'elle apporte	Volume brut (région)
DVF géolocalisé	DGFIP / Etalab	Chaque vente immobilière : date, prix, surface, type, commune, coordonnées	1 393 403 lignes
DPE logements existants	ADEME	Chaque diagnostic énergétique depuis juillet 2021 : étiquette A→G, surface, période de construction	1 279 171 DPE
Comparateur de territoires	INSEE	Pour chaque commune : population, niveau de vie médian, parc de logements	2 925 032 lignes (France)

Le code commune INSEE (5 caractères, par exemple 59350 pour Lille) est présent dans les trois sources : c'est la **clé de jointure** qui permet de tout croiser.

1.2 L'architecture



C'est une architecture **ELT** (Extract, Load, Transform) : on charge d'abord les données telles quelles, puis on les transforme dans l'entrepôt avec dbt. L'inverse, l'ETL, transforme avant de charger (par exemple avec pandas). L'ELT est la norme actuelle : la donnée brute reste disponible, et toutes les règles métier vivent dans dbt, versionnées et testées.

1.3 Les choix structurants, et pourquoi

Choix	Alternative écartée	Raison
Thème logement	Énergie, emploi	Visuel (cartes, prix), multi-sources, et différent de ton projet CO2 Fabric
DuckDB comme entrepôt	Neon seul, BigQuery	Local, gratuit, très rapide en analytique ; DVF + DPE dépasseraient vite l'offre gratuite de Neon
Neon pour Power BI	Lire DuckDB depuis Windows	Power BI (dans Parallels) lit un Postgres cloud sans driver ODBC ni dossier partagé
Mac pour le pipeline	Tout dans Parallels	Python, DuckDB et dbt tournent nativement sur macOS et profitent de toute la RAM ; seul Power BI exige Windows

Choix	Alternative écartée	Raison
dbt Core (ligne de commande)	dbt Cloud	dbt Cloud ne se connecte pas à un fichier DuckDB local
Python sans dépendance pour télécharger	requests, Airbyte	urllib (bibliothèque standard) suffit ; moins de choses à installer
Périmètre MVP régional	France entière	Itérer vite ; passer au national ne demande qu'une liste de départements

1.4 L'arborescence du dépôt

open-data-logement/	
├─ README.md	présentation du projet (vitrine GitHub)
├─ requirements.txt	dépendances Python
├─ .gitignore	ce que Git ne doit jamais envoyer (données, secrets)
├─ .env.example	modèle du fichier de connexion Neon (sans mot de passe)
├─ ingestion/	étapes 1 et 2 : télécharger puis charger dans DuckDB
│ ├─ config.py	
│ ├─ download_dvf.py	
│ ├─ download_insee.py	
│ ├─ download_dpe.py	
│ └─ load_raw.py	
├─ dbt/	étape 3 : le projet dbt
│ ├─ dbt_project.yml	configuration du projet
│ ├─ profiles.yml	connexion à DuckDB
│ ├─ macros/	fonctions Jinja réutilisables
│ ├─ seeds/	petites tables de référence en CSV
│ ├─ models/	
│ │ ├─ staging/	typage des sources
│ │ ├─ intermediate/	règles métier
│ │ └─ marts/	tables finales + schéma en étoile
│ └─ tests/	tests SQL personnalisés
├─ export/export_neon.py	étape 4 : copier les tables finales vers Neon
├─ dashboard/mesures.dax	brouillon des mesures Power BI
└─ data/	(ignoré par Git) fichiers bruts + warehouse.duckdb

1.5 L'entonnoir des données

Voici ce que devient la donnée à chaque couche, pour la région entière. Ces chiffres sont une bonne façon de vérifier qu'un pipeline fait ce qu'on attend :

Étape	DVF (ventes)	DPE (diagnostics)
Brut chargé (raw)	1 393 403 lignes	1 279 171 DPE
Après regroupement par vente / par logement	546 403 mutations	882 664 logements (dernier DPE)
Ventes d'un seul logement, nature « Vente »	355 725 ventes	—
Prix plausible (table de faits finale)	353 702 ventes	442 474 lignes mensuelles agrégées

À retenir

- Une ligne DVF n'est pas une vente : 1,39 million de lignes ne représentent que 546 000 mutations.
- Un DPE n'est pas un logement : 1,28 million de DPE correspondent à 883 000 logements, parce que beaucoup de logements sont rediagnostiqués.
- Comprendre le « grain » de chaque source (ce que représente une ligne) est la compétence clé de l'Analytics Engineer. C'est ce qui évite les chiffres faux.

2. Mettre en place l'environnement

2.1 Où tourne quoi

Ton Mac est un Intel sous macOS 13, avec Windows dans Parallels Desktop. Le pipeline tourne entièrement sur le Mac ; seul Power BI Desktop tourne dans Windows. Les deux se rejoignent par Neon, dans le cloud :

Composant	Où	Pourquoi
Python, DuckDB, dbt, Git	macOS (Terminal)	Natif, rapide, toute la RAM disponible
Neon PostgreSQL	Cloud	Point de rencontre entre le Mac et Windows
Power BI Desktop	Windows (Parallels)	N'existe que sous Windows

2.2 L'environnement virtuel Python

Un environnement virtuel (*venv*) est un dossier qui contient une installation Python isolée, propre au projet. Ça évite de mélanger les versions de bibliothèques entre projets (tu as aussi Anaconda, d'où le (base) dans ton invite de commande).

Terminal (Mac)

```
1 | cd ~/Documents/open-data-logement
2 | python3 -m venv .venv           # crée l'environnement dans le dossier .venv
3 | source .venv/bin/activate       # l'active : l'invite affiche (.venv)
4 | pip install -r requirements.txt  # installe les dépendances listées
```

requirements.txt

```
1 | duckdb>=1.5,<2
2 | dbt-duckdb>=1.9,<2
3 | psycopg[binary]>=3.2,<4
```

Les contraintes `>=1.5,<2` disent « au moins la 1.5, mais pas la 2 » : on accepte les correctifs, pas les changements de version majeure qui peuvent casser le code. `psycopg[binary]` installe la version précompilée du pilote PostgreSQL, qui ne demande aucun compilateur.

Piège rencontré

`pip install -r requirements.txt` a répondu « No such file or directory » : le Terminal était encore dans le sous-dossier `dbt`. Règle : les commandes `python ...` et `git ...` se lancent depuis la racine du projet, les commandes `dbt ...` depuis `dbt/`. La commande `pwd` affiche le dossier courant.

2.3 Git et GitHub

Git garde l'historique de chaque modification ; GitHub héberge ce dépôt en ligne et sert de vitrine pour ton portfolio.

.gitignore

```
1 | # Données (téléchargées ou générées, jamais versionnées)
2 | data/
3 | *.duckdb
4 | *.duckdb.wal
5 | *.part
6 |
7 | # dbt
8 | dbt/target/
9 | dbt/logs/
10 | dbt/dbt_packages/
11 | dbt/.user.yml
12 |
13 | # Secrets
14 | .env
15 |
16 | # Python
17 | .venv/
18 | __pycache__/_
19 | *.pyc
20 |
21 | # macOS
22 | .DS_Store
```

Le **.gitignore** est essentiel : il empêche d'envoyer les centaines de Mo de données (**data/**), les fichiers générés par dbt (**target/**, **logs/**), l'environnement virtuel et surtout le fichier **.env** qui contient ton mot de passe Neon.

Première configuration (une seule fois par machine)

Terminal (Mac)

```
1 | git config --global user.name "Yves-Rolland Boa"
2 | git config --global user.email "yvesboa9@gmail.com" # l'email de ton compte GitHub
3 |
4 | git init -b main # transforme le dossier en dépôt Git
5 | git add . # prépare tous les fichiers (sauf ceux ignorés)
6 | git commit -m "Phase 0-1 : ingestion DVF + modèles dbt"
7 | git remote add origin https://github.com/YvesRollandBoa/open-data-logement.git
8 | git push -u origin main # envoie sur GitHub et mémorise la branche
```

Deux pièges rencontrés

- « unable to auto-detect email address » : Git a besoin de ton nom et email avant le premier commit. L'email doit être celui de GitHub pour que les commits comptent dans ton profil.
- « Password authentication is not supported » : GitHub refuse le mot de passe du compte. On a créé un **jeton d'accès** (github.com/settings/tokens, type classic, cases *repo* et *workflow*) qu'on colle à la place du mot de passe. Le trousseau macOS le mémorise. GitHub CLI via Homebrew a été écarté : sur macOS 13, Homebrew aurait dû tout compiler depuis les sources.

Ensuite, le cycle quotidien se résume à trois commandes :

Terminal (Mac)

```
1 | git add .
2 | git commit -m "Message qui décrit la modification"
3 | git push
```

3. Ingestion : télécharger et charger

Cinq scripts Python dans le dossier `ingestion/`. Quatre téléchargent, un charge. Ils partagent une configuration commune et suivent les mêmes principes :

- **Idempotence** : relancer un script ne refait pas ce qui est déjà fait et ne crée pas de doublon.
- **Écriture atomique** : on écrit dans un fichier temporaire `.part`, renommé seulement une fois complet. Un téléchargement interrompu ne laisse jamais un faux fichier « fini ».
- **Relances automatiques** en cas d'erreur réseau, avec une attente qui augmente à chaque essai.
- **Aucune transformation** : les données arrivent telles quelles dans DuckDB. Le nettoyage appartient à dbt.

Ordre d'exécution (depuis la racine du projet)

```
python ingestion/download_dvf.py      # ventes immobilières (~40 Mo, 25 fichiers)
python ingestion/download_insee.py    # indicateurs communaux (~10 Mo, 1 fichier)
python ingestion/download_dpe.py      # diagnostics énergétiques (~50 Mo, le plus long)
python ingestion/load_raw.py          # charge tout dans data/warehouse.duckdb
```

3.1 config.py : les paramètres partagés

ingestion/config.py

```
1  """Paramètres partagés par les scripts d'ingestion."""
2  from pathlib import Path
3
4  ROOT = Path(__file__).resolve().parents[1]
5  RAW_DIR = ROOT / "data" / "raw"
6  WAREHOUSE = ROOT / "data" / "warehouse.duckdb"
7
8  # MVP : Hauts-de-France. Passer à la France entière une fois le pipeline stable.
9  # (DVF ne couvre ni l'Alsace-Moselle – 57, 67, 68 – ni Mayotte – 976.)
10 DEPARTEMENTS = ["02", "59", "60", "62", "80"]
11
12 # Millésimes disponibles dans geo-dvf (mise à jour d'avril 2026 : 2021 → 2025).
13 ANNEES_DVF = [2021, 2022, 2023, 2024, 2025]
14
15 DVF_URL = "https://files.data.gouv.fr/geo-dvf/latest/csv/{annee}/departements/{dep}.csv.gz"
16
17 # DPE logements existants (ADEME, depuis juillet 2021), via l'API data-fair.
18 DPE_API = "https://data.ademe.fr/data-fair/api/v1/datasets/dpe03existant/lines"
19 DPE_CHAMPS = [
20     "numero_dpe",
21     "numero_dpe_remplace",
22     "date_etablissement_dpe",
23     "code_insee_ban",
24     "nom_commune_ban",
25     "code_departement_ban",
26     "identifiant_ban",
27     "type_batiment",
28     "periode_construction",
29     "annee_construction",
30     "surface_habitable_logement",
31     "etiquette_dpe",
32     "etiquette_ges",
33     "conso_5_usages_par_m2_ep",
34     "emission_ges_5_usages_par_m2",
35 ]
36
37 # INSEE – Base du comparateur de territoires (population, revenus, logement... par commune).
38 INSEE_COMPARATEUR_URL = "https://www.insee.fr/fr/statistiques/fichier/2521169/comparateur.parquet"
```

- `Path(__file__).resolve().parents[1]` calcule la racine du projet à partir de l'emplacement du fichier. Les scripts fonctionnent ainsi quel que soit le dossier d'où on les lance.
- Les départements sont des **textes** ("02"), jamais des nombres : 02 deviendrait 2, et la Corse s'écrit 2A / 2B.
- Passer à toute la France ne demande que de modifier **DEPARTEMENTS** (et la variable `dbt` équivalente).
- **DPE_CHAMPS** liste les 15 colonnes demandées à l'API parmi plus de 200 : on ne télécharge que l'utile.

3.2 download_dvf.py : les ventes immobilières

Etalab publie DVF géolocalisé sous forme d'un fichier compressé par année et par département, à une adresse prévisible : `.../geo-dvf/latest/csv/2024/departements/59.csv.gz`. Le script boucle sur 5 années × 5 départements.

ingestion/download_dvf.py

```

1  """Télécharge les fichiers DVF géolocalisés (geo-dvf, Etalab) par année et département.
2
3  Usage :
4      python ingestion/download_dvf.py                # config par défaut
5      python ingestion/download_dvf.py --deps 59 62 --annees 2024 2025
6      python ingestion/download_dvf.py --force        # re-télécharge tout
7
8  Les fichiers déjà présents sont ignorés (téléchargement idempotent).
9  """
10 import argparse
11 import sys
12 import time
13 import urllib.error
14 import urllib.request
15
16 from config import ANNEES_DVF, DEPARTEMENTS, DVF_URL, RAW_DIR
17
18 DEST = RAW_DIR / "dvf"
19
20
21 def telecharger(url: str, cible, tentatives: int = 3) -> bool:
22     tmp = cible.with_suffix(cible.suffix + ".part")
23     for essai in range(1, tentatives + 1):
24         try:
25             with urllib.request.urlopen(url, timeout=120) as rep, open(tmp, "wb") as f:
26                 while bloc := rep.read(1 << 20):
27                     f.write(bloc)
28             tmp.rename(cible)
29             return True
30         except urllib.error.HTTPError as e:
31             if e.code == 404:
32                 print(f" absent (404) : {url}")
33                 return False
34             print(f" HTTP {e.code}, essai {essai}/{tentatives}")
35         except (urllib.error.URLError, TimeoutError) as e:
36             print(f" erreur réseau ({e}), essai {essai}/{tentatives}")
37             time.sleep(2 * essai)
38     tmp.unlink(missing_ok=True)
39     return False
40
41
42 def main() -> int:
43     p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFo
44         rmatter)
45     p.add_argument("--deps", nargs="+", default=DEPARTEMENTS)
46     p.add_argument("--annees", nargs="+", type=int, default=ANNEES_DVF)
47     p.add_argument("--force", action="store_true")
48     args = p.parse_args()
49
50     echecs = []
51     for annee in args.annees:
52         for dep in args.deps:
53             cible = DEST / str(annee) / f"{dep}.csv.gz"
54             cible.parent.mkdir(parents=True, exist_ok=True)
55             if cible.exists() and not args.force:

```

```

55 |         print(f"[skip] {annee}/{dep}")
56 |         continue
57 |         print(f"[get ] {annee}/{dep}")
58 |         if not telecharger(DVF_URL.format(annee=annee, dep=dep), cible):
59 |             echecs.append(f"{annee}/{dep}")
60 |
61 |     if echecs:
62 |         print(f"\n{len(echecs)} fichier(s) non récupéré(s) : {' ', '.join(echecs)}")
63 |         return 1
64 |     print("\nTéléchargement DVF terminé.")
65 |     return 0
66 |
67 |
68 | if __name__ == "__main__":
69 |     sys.exit(main())

```

Lignes	Ce qui se passe
21-39	Fonction telecharger : ouvre l'URL et copie la réponse par blocs de 1 Mo (1 << 20 octets) pour ne jamais charger tout le fichier en mémoire. L'opérateur := (le « morse ») lit un bloc et teste s'il est vide dans la même expression.
28	Le fichier .part n'est renommé qu'une fois entièrement écrit.
30-33	Une erreur 404 veut dire « ce fichier n'existe pas » : inutile de réessayer.
35-37	Erreur réseau : nouvel essai après 2, 4 puis 6 secondes.
54-56	Si le fichier existe déjà et que --force n'est pas passé, on le saute.
61-65	Code de sortie 1 si un fichier manque : utile plus tard pour l'automatisation (GitHub Actions saura que l'étape a échoué).

argparse ajoute des options en ligne de commande, par exemple **python ingestion/download_dvf.py --deps 59 --annees 2025** pour ne récupérer qu'un fichier.

3.3 download_insee.py : les indicateurs communaux

ingestion/download_insee.py

```

1 | """Télécharge la base du comparateur de territoires de l'INSEE (fichier Parquet, ~9 Mo).
2 |
3 | Format « long » : une ligne par territoire x indicateur x période
4 | (colonnes GEO, TAB_MEASURE, TIME_PERIOD, OBS_VALUE...). Le pivot se fait dans dbt.
5 |
6 | Usage :
7 |     python ingestion/download_insee.py [--force]
8 |
9 | import argparse
10 | import sys
11 | import urllib.request
12 |
13 | from config import INSEE_COMPARATEUR_URL, RAW_DIR
14 |
15 | CIBLE = RAW_DIR / "insee" / "comparateur.parquet"
16 |
17 |
18 | def main() -> int:
19 |     p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFo
20 |         rmatter)
21 |     p.add_argument("--force", action="store_true")
22 |     args = p.parse_args()
23 |
24 |     if CIBLE.exists() and not args.force:
25 |         print("[skip] INSEE comparateur")
26 |         return 0
27 |     CIBLE.parent.mkdir(parents=True, exist_ok=True)
28 |     print("[get ] INSEE comparateur")
29 |     tmp = CIBLE.with_suffix(".part")
30 |     req = urllib.request.Request(INSEE_COMPARATEUR_URL, headers={"User-Agent": "open-data-logement

```

```

    })
30     with urllib.request.urlopen(req, timeout=120) as rep, open(tmp, "wb") as f:
31         f.write(rep.read())
32         tmp.rename(CIBLE)
33         print(f"      {CIBLE.stat().st_size / 1e6:.1f} Mo écrits")
34         return 0
35
36
37 if __name__ == "__main__":
38     sys.exit(main())

```

Le plus simple des scripts : un seul fichier Parquet d'environ 10 Mo. **Parquet** est un format colonnaire compressé qui garde les types (nombres, textes) : pas besoin de tout lire en texte comme pour les CSV. L'en-tête **User-Agent** identifie poliment le programme auprès du serveur INSEE.

Découverte en cours de route

L'INSEE a changé le format de ce fichier : il est désormais « long », avec une ligne par commune × indicateur × année (colonnes **GEO**, **TAB_MEASURE**, **TIME_PERIOD**, **OBS_VALUE**). La page de téléchargement ne listait pas les codes des indicateurs. Plutôt que de les deviner, on a d'abord téléchargé le fichier, puis lu la liste réelle des codes (**POP** = population, **MED_SL** = niveau de vie médian, **DWELLINGS** = logements...) avant d'écrire le modèle dbt. Règle générale : regarder la donnée avant de coder dessus.

3.4 download_dpe.py : les diagnostics via une API paginée

Les DPE ne sont pas disponibles en fichier par département : on interroge l'API *data-fair* de l'ADEME. Une API ne renvoie jamais 600 000 lignes d'un coup : elle les découpe en **pages**. Chaque réponse contient une page de résultats et un lien **next** vers la suivante. On suit ces liens jusqu'au bout.

ingestion/download_dpe.py

```

1  """Télécharge les DPE des logements existants (ADEME) par département via l'API data-fair.
2
3  L'API renvoie les lignes par pages ; on suit le lien `next` jusqu'au bout.
4  Chaque département est écrit dans data/raw/dpe/<dep>.ndjson.gz (une ligne JSON par DPE).
5
6  Reprise sur incident : après chaque page, l'URL de la page suivante est sauvegardée
7  dans <dep>.next. Si le script est interrompu (coupure réseau, Ctrl+C), il suffit de
8  le relancer : il repart de la dernière page téléchargée au lieu de tout refaire.
9
10 Usage :
11     python ingestion/download_dpe.py
12     python ingestion/download_dpe.py --deps 59 --force
13
14 Compter quelques minutes par département (le Nord seul dépasse 600 000 DPE).
15 """
16 import argparse
17 import gzip
18 import json
19 import sys
20 import time
21 import urllib.error
22 import urllib.parse
23 import urllib.request
24
25 from config import DEPARTEMENTS, DPE_API, DPE_CHAMPS, RAW_DIR
26
27 DEST = RAW_DIR / "dpe"
28 TAILLE_PAGE = 5_000 # pages plus petites = moins de timeouts côté API
29
30
31 def get_json(url: str, tentatives: int = 6) -> dict:
32     for essai in range(1, tentatives + 1):
33         try:
34             with urllib.request.urlopen(url, timeout=180) as rep:
35                 return json.load(rep)
36         except urllib.error.HTTPError as e:
37             # 429 = trop de requêtes : on attend plus longtemps

```

```

38         attente = 30 if e.code == 429 else 5 * essai
39         print(f"\n HTTP {e.code}, nouvel essai dans {attente}s ({essai}/{tentatives})")
40         time.sleep(attente)
41     except (urllib.error.URLError, TimeoutError, ConnectionError) as e:
42         print(f"\n erreur réseau ({e}), essai {essai}/{tentatives}")
43         time.sleep(5 * essai)
44     raise RuntimeError(f"Échec après {tentatives} essais – relancer le script pour reprendre.")
45
46
47 def telecharger_departement(dep: str) -> int:
48     cible = DEST / f"{dep}.ndjson.gz"
49     partiel = DEST / f"{dep}.ndjson.gz.part"
50     reprise = DEST / f"{dep}.next"
51
52     if partiel.exists() and reprise.exists():
53         url = reprise.read_text().strip()
54         with gzip.open(partiel, "rt", encoding="utf-8") as f:
55             n = sum(1 for _ in f)
56         print(f" reprise après {n:,} lignes")
57     else:
58         params = {
59             "qs": f"code_departement_ban:{dep}",
60             "select": ", ".join(DPE_CHAMPS),
61             "size": TAILLE_PAGE,
62         }
63         url = f"{DPE_API}?{urllib.parse.urlencode(params)}"
64         partiel.unlink(missing_ok=True)
65         n = 0
66
67     while url:
68         page = get_json(url)
69         if n == 0 and isinstance(page.get("total"), int):
70             print(f" {page['total']:,} DPE annoncés")
71         # Mode "a" : chaque page devient un membre gzip distinct, relu sans problème.
72         with gzip.open(partiel, "at", encoding="utf-8") as f:
73             for ligne in page.get("results", []):
74                 ligne.pop("_score", None)
75                 f.write(json.dumps(ligne, ensure_ascii=False) + "\n")
76                 n += 1
77         url = page.get("next")
78         if url:
79             reprise.write_text(url)
80         print(f" {n:,} lignes", end="\r")
81
82     partiel.rename(cible)
83     reprise.unlink(missing_ok=True)
84     print(f" {n:,} lignes écrites")
85     return n
86
87
88 def main() -> int:
89     p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpF
90                                ormatter)
91     p.add_argument("--deps", nargs="+", default=DEPARTEMENTS)
92     p.add_argument("--force", action="store_true")
93     args = p.parse_args()
94
95     DEST.mkdir(parents=True, exist_ok=True)
96     echecs = []
97     for dep in args.deps:
98         if args.force:
99             for suffixe in (".ndjson.gz", ".ndjson.gz.part", ".next"):
100                 (DEST / f"{dep}{suffixe}").unlink(missing_ok=True)
101             if (DEST / f"{dep}.ndjson.gz").exists():
102                 print(f"[skip] DPE {dep}")
103                 continue
104             print(f"[get ] DPE {dep}")
105             try:
106                 telecharger_departement(dep)
107             except RuntimeError as e:
108                 print(f"\n {e}")
109                 echecs.append(dep)
110
111     if echecs:
112         print(f"\nDépartements incomplets : {' ', ' '.join(echecs)} – relancer le script pour reprend
113 re.")

```

```

112 |         return 1
113 |     print("\nTéléchargement DPE terminé.")
114 |     return 0
115 |
116 |
117 | if __name__ == "__main__":
118 |     sys.exit(main())

```

Lignes	Ce qui se passe
28	5 000 lignes par page. La première version en demandait 10 000 et subissait des timeouts.
31-44	get_json : 6 essais. Le code HTTP 429 signifie « trop de requêtes » : on attend alors 30 secondes.
58-63	Première page : filtre qs=code_departement_ban:59 , colonnes voulues, taille de page. urlencode encode proprement les caractères spéciaux de l'URL.
67-80	Boucle de pagination : lire la page, écrire chaque ligne au format NDJSON (un objet JSON par ligne), passer au lien next .
78-79	Point de reprise : après chaque page, l'URL suivante est sauvegardée dans 59.next .
52-56	Au redémarrage, si .part et .next existent, on repart de la dernière page au lieu de tout refaire.
72	Ouverture en mode ajout (" at ") : chaque page ajoute un bloc gzip au fichier. Un fichier gzip fait de plusieurs blocs se relit comme un seul.

Pourquoi la reprise a été ajoutée

Pendant le téléchargement du Nord puis du Pas-de-Calais, l'API de l'ADEME est devenue très lente (130 000 lignes en 2 h 30, contre 150 000 en 2 minutes pour l'Oise). La première version ne savait pas reprendre : une coupure obligeait à tout recommencer. On a aussi conseillé **caffeinate -i python ingestion/download_dpe.py** : la commande macOS **caffeinate** empêche la mise en veille pendant l'exécution.

3.5 load_raw.py : charger dans DuckDB

DuckDB est une base de données analytique « embarquée » : pas de serveur, tout tient dans un fichier (**data/warehouse.duckdb**). Elle sait lire directement des CSV, du JSON et du Parquet, même compressés.

ingestion/load_raw.py

```

1 | """Charge les fichiers bruts dans DuckDB (schéma `raw`), sans transformation.
2 |
3 | Les colonnes des fichiers CSV/JSON sont lues en texte : le typage se fait dans dbt
4 | (staging), ce qui évite de perdre les zéros des codes (ex. département "02",
5 | commune "02001") et garde la couche raw fidèle à la source.
6 |
7 | Chaque source est chargée si ses fichiers sont présents, ignorée sinon.
8 |
9 | Usage :
10 |     python ingestion/load_raw.py
11 | """
12 | import sys
13 |
14 | import duckdb
15 |
16 | from config import RAW_DIR, WAREHOUSE
17 |
18 |
19 | def charger_dvf(con: duckdb.DuckDBPyConnection) -> bool:
20 |     fichiers = sorted((RAW_DIR / "dvf").glob("*/*.csv.gz"))
21 |     if not fichiers:
22 |         return False
23 |     motif = str(RAW_DIR / "dvf" / "*" / "*.csv.gz")
24 |     con.execute(f"""
25 |         create or replace table raw.dvf_mutations as
26 |         select

```

```

27         *,
28         regexp_extract(filename, '(\d{4})[/\\\[^\d\\]+$', 1) as _annee_fichier,
29         current_timestamp as _loaded_at
30     from read_csv(
31         '{motif}',
32         header = true,
33         all_varchar = true,
34         union_by_name = true,
35         filename = true
36     )
37 """
38 compter(con, "raw.dvf_mutations", len(fichiers))
39 return True
40
41
42 def charger_dpe(con: duckdb.DuckDBPyConnection) -> bool:
43     fichiers = sorted((RAW_DIR / "dpe").glob("*.ndjson.gz"))
44     if not fichiers:
45         return False
46     motif = str(RAW_DIR / "dpe" / "*.ndjson.gz")
47     con.execute(f"""
48         create or replace table raw.dpe_logements as
49         select
50             *,
51             current_timestamp as _loaded_at
52     from read_json(
53         '{motif}',
54         format = 'newline_delimited',
55         union_by_name = true,
56         -- tout en texte, comme pour DVF : le typage est fait par dbt
57         columns = {{
58             numero_dpe: 'VARCHAR', numero_dpe_remplace: 'VARCHAR',
59             date_etablissement_dpe: 'VARCHAR', code_insee_ban: 'VARCHAR',
60             nom_commune_ban: 'VARCHAR', code_departement_ban: 'VARCHAR',
61             identifiant_ban: 'VARCHAR', type_batiment: 'VARCHAR',
62             periode_construction: 'VARCHAR', annee_construction: 'VARCHAR',
63             surface_habitable_logement: 'VARCHAR', etiquette_dpe: 'VARCHAR',
64             etiquette_ges: 'VARCHAR', conso_5_usages_par_m2_ep: 'VARCHAR',
65             emission_ges_5_usages_par_m2: 'VARCHAR'
66         }}
67     )
68 """
69     compter(con, "raw.dpe_logements", len(fichiers))
70     return True
71
72
73 def charger_insee(con: duckdb.DuckDBPyConnection) -> bool:
74     fichier = RAW_DIR / "insee" / "comparateur.parquet"
75     if not fichier.exists():
76         return False
77     # Parquet : les types sont déjà définis par l'INSEE, on garde tel quel.
78     con.execute(f"""
79         create or replace table raw.insee_comparateur as
80         select *, current_timestamp as _loaded_at
81     from read_parquet('{fichier}')
82 """
83     )
84     compter(con, "raw.insee_comparateur", 1)
85     return True
86
87 def compter(con, table: str, nb_fichiers: int) -> None:
88     n = con.execute(f"select count(*) from {table}").fetchone()[0]
89     print(f"{table:<24} {n:>12,} lignes  ({nb_fichiers} fichier(s))")
90
91
92 def main() -> int:
93     WAREHOUSE.parent.mkdir(parents=True, exist_ok=True)
94     with duckdb.connect(str(WAREHOUSE)) as con:
95         con.execute("create schema if not exists raw")
96         charges = {
97             "DVF": charger_dvf(con),
98             "DPE": charger_dpe(con),
99             "INSEE": charger_insee(con),
100         }
101     manquants = [nom for nom, ok in charges.items() if not ok]
102     if manquants:

```

```

103 |         print(f"Non chargé (fichiers absents) : {' , '.join(manquants)}")
104 |     if not any(charges.values()):
105 |         print("Aucune donnée : lance d'abord les scripts download_*.py")
106 |         return 1
107 |     return 0
108 |
109 |
110 | if __name__ == "__main__":
111 |     sys.exit(main())

```

Lignes	Ce qui se passe
24-37	Un seul <code>read_csv</code> lit les 25 fichiers DVF grâce au motif <code>*/*.csv.gz</code> . <code>union_by_name</code> aligne les colonnes par nom si un fichier en a une de plus ou de moins. <code>filename = true</code> ajoute le chemin du fichier d'origine.
33	<code>all_varchar = true</code> : toutes les colonnes en texte. Sinon DuckDB devinerait les types et transformerait le département « 02 » en nombre 2, cassant les jointures futures.
28	Une expression régulière extrait l'année du chemin du fichier. Les doubles accolades <code>{4}</code> sont nécessaires parce que la requête est dans une f-string Python.
29	<code>_loaded_at</code> horodate le chargement : dbt s'en sert pour vérifier la fraîcheur des sources.
43-70	DPE : <code>read_json</code> au format NDJSON, avec les types forcés en texte colonne par colonne.
73-84	INSEE : le Parquet est déjà typé par l'INSEE, on le garde tel quel.
92-105	Chaque source n'est chargée que si ses fichiers sont là, ce qui permet de tester une source à la fois.

À retenir

- `create or replace table` rend le chargement idempotent : relancer le script remplace la table au lieu d'ajouter des doublons.
- La couche `raw` est une photocopie fidèle de la source. Si une règle métier change, on ne retélécharge rien : on relance dbt.
- `load_raw.py` ignore les fichiers `.part` en cours : on peut charger et tester pendant qu'un téléchargement continue.

4. Transformation avec dbt

dbt (data build tool) transforme des données déjà chargées dans un entrepôt, avec du SQL. Chaque fichier `.sql` du dossier `models/` est un **modèle** : une requête `select` que dbt matérialise en vue ou en table. dbt déduit l'ordre d'exécution des dépendances entre modèles, exécute les tests et génère la documentation.

Tu connais déjà dbt Cloud avec ton projet RH. Ici, on utilise **dbt Core** en ligne de commande, avec l'adaptateur `dbt-duckdb`. Le SQL et la logique sont les mêmes ; seule la connexion change.

4.1 Les trois couches

Couche	Rôle	Matérialisation	Règle
staging (<code>stg_</code>)	Typage, renommer, une ligne source = une ligne	vue	Aucune règle métier
intermediate (<code>int_</code>)	Appliquer les règles métier, changer le grain	table	Pas destiné à l'utilisateur final
marts (<code>mart_</code> , <code>dim_</code> , <code>fct_</code>)	Tables prêtes à consommer	table	Stables, documentées, testées

Les vues de staging ne stockent rien : elles sont recalculées à chaque lecture, ce qui convient à un simple typage. Les couches suivantes sont des tables, car leurs calculs (regroupements, médianes) coûtent cher.

4.2 Configuration du projet

`dbt/dbt_project.yml`

```

1 | name: open_data_logement
2 | version: "0.1.0"
3 | config-version: 2
4 | profile: open_data_logement
5 |
6 | model-paths: ["models"]
7 | test-paths: ["tests"]
8 | seed-paths: ["seeds"]
9 | target-path: "target"
10 | clean-targets: ["target", "dbt_packages"]
11 |
12 | models:
13 |   open_data_logement:
14 |     staging:
15 |       +schema: staging
16 |       +materialized: view
17 |     intermediate:
18 |       +schema: intermediate
19 |       +materialized: table
20 |     marts:
21 |       +schema: marts
22 |       +materialized: table
23 |
24 | seeds:
25 |   open_data_logement:
26 |     +schema: referentiel
27 |
28 | vars:
29 |   # Périmètre géographique (doit correspondre à DEPARTEMENTS dans ingestion/config.py)
30 |   departements: ["02", "59", "60", "62", "80"]
31 |   # Années DVF regroupées pour les indicateurs « récents » (2 ans = assez de ventes par commune)
32 |   annees_recentes: [2024, 2025]
33 |
34 | flags:

```

```
35 | send_anonymous_usage_stats: false
```

- **+schema** range chaque couche dans son propre schéma DuckDB (**staging**, **intermediate**, **marts**, **referentiel**).
- **vars** définit des variables réutilisables dans le SQL avec **var('departements')**. Changer de périmètre ou d'années ne touche qu'à ce fichier.
- **send_anonymous_usage_stats: false** coupe l'envoi de statistiques d'usage à dbt Labs.

dbt/profiles.yml

```
1 | open_data_logement:
2 |   target: dev
3 |   outputs:
4 |     dev:
5 |       type: duckdb
6 |       # Chemin relatif au dossier dbt/ (lancer dbt depuis ce dossier)
7 |       path: ../data/warehouse.duckdb
8 |       threads: 4
```

Le profil indique à dbt où se connecter. Avec DuckDB, c'est un simple chemin de fichier, relatif au dossier **dbt/**. C'est pour ça qu'on lance **dbt build --profiles-dir .** : l'option dit à dbt de chercher **profiles.yml** dans le dossier courant plutôt que dans **~/dbt/**.

Les macros

Une macro est une fonction écrite en Jinja (le langage de gabarits de dbt) qui génère du SQL.

dbt/macros/generate_schema_name.sql

```
1 | {%- Utilise le nom de schéma tel quel (staging, intermediate, marts)
2 |   au lieu du préfixe par défaut de dbt (main_staging, ...). -#}
3 | {% macro generate_schema_name(custom_schema_name, node) -%}
4 |   {%- if custom_schema_name is none -%}{{ target.schema }}{%- else -%}{{ custom_schema_name | tri
5 |     m }}{%- endif -%}
6 | {%- endmacro %}
```

Par défaut, dbt préfixe les schémas personnalisés par le schéma de la connexion : on obtiendrait **main_staging**. Redéfinir cette macro spéciale de dbt garde les noms tels quels.

dbt/macros/liste_departements.sql

```
1 | {%- Rend la var `departements` sous forme de liste SQL : '02', '59', ... -#}
2 | {% macro liste_departements() -%}
3 |   {{ "'" ~ var('departements') | join(", ") ~ "'" }}
4 | {%- endmacro %}
```

Transforme la liste **["02", "59"]** en texte SQL **'02', '59'**, utilisable dans **where code_departement in (...)**. Écrivez une fois, utilisée dans cinq modèles.

dbt/macros/test_unique_combinaison.sql

```
1 | {%- Test générique : la combinaison de colonnes est unique (grain de la table). -#}
2 | {% test unique_combinaison(model, colonnes) %}
3 | select {{ colonnes | join(', ') }}, count(*) as nb
4 | from {{ model }}
5 | group by {{ colonnes | join(', ') }}
6 | having count(*) > 1
7 | {% endtest %}
```

Un **test générique** maison : il renvoie les combinaisons de colonnes présentes plus d'une fois. Un test dbt échoue dès que sa requête renvoie au moins une ligne. On l'utilise pour vérifier le grain de **fct_dpe_mensuel**.

4.3 Les sources

dbt/models/staging/_sources.yml

```
1 | version: 2
2 |
3 | sources:
4 |   - name: raw
5 |     description: Données brutes chargées par ingestion/load_raw.py (sans transformation).
6 |     tables:
7 |       - name: dvf_mutations
8 |         description: >
9 |           DVF géolocalisé (geo-dvf, Etalab / DGFIP). Une ligne par disposition × local × parcelle
10 |           :
11 |             une même mutation (vente) peut donc apparaître sur plusieurs lignes, avec la valeur
12 |             foncière répétée sur chacune.
13 |         loaded_at_field: _loaded_at
14 |         columns:
15 |           - name: id_mutation
16 |             tests: [not_null]
17 |
18 |       - name: dpe_logements
19 |         description: >
20 |           DPE des logements existants (ADEME, depuis juillet 2021), toutes colonnes en texte.
21 |           Un même logement peut avoir plusieurs DPE (nouveau diagnostic, correction).
22 |         loaded_at_field: _loaded_at
23 |         columns:
24 |           - name: numero_dpe
25 |             tests:
26 |               - not_null
27 |                 # Doublons techniques possibles si l'ADEME met à jour pendant la pagination
28 |                 # de l'API : simple alerte, ils sont retirés en staging.
29 |               - unique:
30 |                 config:
31 |                   severity: warn
32 |
33 |       - name: insee_comparateur
34 |         description: >
35 |           Base du comparateur de territoires INSEE (géographie au 1er janvier 2026), format long :
36 |           une ligne par territoire × indicateur (TAB_MEASURE) × période (TIME_PERIOD).
37 |           CONF_STATUS = 'C' signale une valeur couverte par le secret statistique.
38 |         loaded_at_field: _loaded_at
```

Une **source** déclare une table chargée hors de dbt (ici par `load_raw.py`). Les modèles la lisent avec `{{ source('raw', 'dvf_mutations') }}` : dbt connaît alors l'origine de chaque donnée et l'affiche dans le graphe de lignage de la documentation.

- Des tests peuvent porter directement sur les sources, avant toute transformation.
- **severity: warn** transforme un échec en simple alerte : un doublon technique de téléchargement DPE ne bloque pas la chaîne, puisqu'il est retiré en staging.
- **loaded_at_field** désigne la colonne d'horodatage, pour la commande `dbt source freshness`.

4.4 Staging : typer les sources

dbt/models/staging/stg_dvf_mutations.sql

```
1 | -- Typage et renommage du DVF brut. Aucune règle métier ici : une ligne source = une ligne.
2 | with source as (
3 |   select * from {{ source('raw', 'dvf_mutations') }}
4 | )
5 |
6 | select
7 |   id_mutation,
8 |   cast(date_mutation as date) as date_mutation,
9 |   year(cast(date_mutation as date)) as annee,
10 |  nature_mutation,
11 |  try_cast(replace(valeur_fonciere, ',', '.')) as valeur_fonciere,
12 |  code_postal,
```

```

13 |     code_commune,
14 |     nom_commune,
15 |     code_departement,
16 |     id_parcelle,
17 |     lot1_numero,
18 |     try_cast(nombre_lots as integer)           as nombre_lots,
19 |     code_type_local,
20 |     type_local,
21 |     try_cast(surface_reelle_bati as double)    as surface_reelle_bati,
22 |     try_cast(nombre_pieces_principales as integer) as nombre_pieces_principales,
23 |     try_cast(surface_terrain as double)        as surface_terrain,
24 |     try_cast(longitude as double)             as longitude,
25 |     try_cast(latitude as double)              as latitude,
26 |     _loaded_at
27 | from source

```

- `try_cast` convertit le texte en nombre, et renvoie NULL au lieu d'une erreur si la valeur est invalide. Un `cast` simple ferait échouer tout le modèle sur une seule valeur sale.
- DVF écrit les montants avec une virgule décimale (« 185000,00 ») : `replace(',', ' ', '.')` la convertit avant le typage.
- On calcule `annee` une fois ici, plutôt que dans chaque modèle en aval.

dbt/models/staging/stg_dpe_logements.sql

```

1 | -- Typage du DPE brut. Un DPE = une ligne (doublons techniques de téléchargement retirés).
2 | with source as (
3 |     select * from {{ source('raw', 'dpe_logements') }}
4 | )
5 |
6 | select
7 |     numero_dpe,
8 |     numero_dpe_remplace,
9 |     try_cast(date_etablissement_dpe as date)           as date_etablissement_dpe,
10 |     code_insee_ban                                   as code_commune,
11 |     nom_commune_ban                                  as nom_commune,
12 |     code_departement_ban                             as code_departement,
13 |     identifiant_ban,
14 |     lower(type_batiment)                             as type_batiment,
15 |     periode_construction,
16 |     try_cast(annee_construction as integer)           as annee_construction,
17 |     try_cast(surface_habitable_logement as double)    as surface_habitable,
18 |     upper(etiquette_dpe)                             as etiquette_dpe,
19 |     upper(etiquette_ges)                             as etiquette_ges,
20 |     try_cast(conso_5_usages_par_m2_ep as double)      as conso_energie_primaire_m2,
21 |     try_cast(emission_ges_5_usages_par_m2 as double)  as emission_ges_m2,
22 |     _loaded_at
23 | from source
24 | qualify row_number() over (partition by numero_dpe order by _loaded_at) = 1

```

Même principe, plus deux normalisations (`lower` sur le type de bâtiment, `upper` sur les étiquettes). La clause `qualify` filtre sur le résultat d'une fonction de fenêtre : pour chaque `numero_dpe`, elle ne garde que la première occurrence, ce qui retire les doublons techniques éventuels du téléchargement. C'est la façon DuckDB (comme Snowflake ou BigQuery) d'écrire en une ligne ce qui demanderait sinon une sous-requête avec `row_number()`.

dbt/models/staging/stg_insee_indicateurs.sql

```

1 | -- Indicateurs INSEE au niveau commune, format long, renommés en français.
2 | -- Les valeurs sous secret statistique (CONF_STATUS = 'C') sont mises à NULL
3 | -- pour qu'aucune ne soit utilisée par erreur dans un calcul.
4 | with source as (
5 |     select * from {{ source('raw', 'insee_comparateur') }}
6 |     where GEO_OBJECT = 'COM'
7 | )
8 |
9 | select
10 |     GEO                                   as code_commune,
11 |     GEO_LABEL                             as nom_commune,

```

```

12 |     TAB_MEASURE                                as indicateur,
13 |     TAB_MEASURE_LABEL                          as libelle_indicateur,
14 |     cast(TIME_PERIOD as integer)               as annee,
15 |     case when CONF_STATUS = 'C' then null else OBS_VALUE end as valeur,
16 |     coalesce(CONF_STATUS = 'C', false)        as est_secret_statistique,
17 |     GEO_REF                                    as millesime_geographie
18 | from source

```

- Ne garde que le niveau commune (**GEO_OBJECT** = 'COM') : le fichier contient aussi régions, départements, intercommunalités...
- **Secret statistique** : quand **CONF_STATUS** = 'C', l'INSEE ne doit pas diffuser la valeur. On la remplace par NULL, pour qu'aucun calcul ne l'utilise par erreur, et on garde un indicateur booléen.

dbt/models/staging/_staging.yml

```

1 | version: 2
2 |
3 | models:
4 |   - name: stg_dvf_mutations
5 |     description: DVF typé et renommé, grain identique à la source.
6 |     columns:
7 |       - name: id_mutation
8 |         tests: [not_null]
9 |       - name: date_mutation
10 |        tests: [not_null]
11 |       - name: code_commune
12 |        tests: [not_null]
13 |
14 |   - name: stg_dpe_logements
15 |     description: DPE typés et renommés, un DPE par ligne.
16 |     columns:
17 |       - name: numero_dpe
18 |         tests: [unique, not_null]
19 |       - name: code_commune
20 |         tests: [not_null]
21 |       - name: etiquette_dpe
22 |         tests:
23 |           - accepted_values:
24 |             arguments:
25 |               values: ["A", "B", "C", "D", "E", "F", "G"]
26 |       - name: type_batiment
27 |         tests:
28 |           - accepted_values:
29 |             arguments:
30 |               values: ["maison", "appartement", "immeuble"]
31 |
32 |   - name: stg_insee_indicateurs
33 |     description: Indicateurs communaux INSEE (format long), secret statistique mis à NULL.
34 |     columns:
35 |       - name: code_commune
36 |         tests: [not_null]
37 |       - name: indicateur
38 |         tests: [not_null]

```

Les tests déclaratifs : **not_null**, **unique** et **accepted_values**. Depuis dbt 1.10, les paramètres d'un test générique se placent sous **arguments** (l'ancienne syntaxe provoquait un avertissement de dépréciation).

Piège rencontré

Un test **accepted_values** sur **nature_mutation** plantait : la valeur « Vente en l'état futur d'achèvement » contient des apostrophes, que dbt n'échappe pas dans le SQL généré. On l'a remplacé par un test singulier écrit à la main (voir 4.8), où l'apostrophe est doublée (**l' 'état**), la syntaxe SQL standard.

4.5 Intermediate : les règles métier

int_ventes_logement : d'une ligne DVF à une vente

C'est le modèle le plus important du projet. Dans DVF, une ligne correspond à une disposition × un local × une parcelle. Un appartement vendu avec une cave et un parking produit donc trois lignes, et le **prix total de la vente est répété sur chacune**. Une moyenne directe compterait ce prix trois fois.

dbt/models/intermediate/int_ventes_logement.sql

```
1  /*
2   Une ligne = une vente d'UN seul logement (maison ou appartement).
3
4   Règles (méthode proche de celle du Cerema / DVF+) :
5   - nature « Vente » uniquement (hors VEFA, adjudications, échanges...) ;
6   - la mutation contient exactement un logement : les ventes en bloc (immeubles,
7     lots multiples) ont un prix global impossible à ramener au m² d'un logement ;
8   - pas de local d'activité dans la mutation (sinon le prix mélange les usages) ;
9   - le logement a une surface renseignée (sinon pas de prix au m²) ;
10  - les dépendances (garage, cave...) sont tolérées : elles restent dans le prix,
11    comme sur le marché réel.
12  Les valeurs aberrantes sont filtrées ensuite (voir `est_prix_plausible`).
13  */
14  with lignes as (
15      select * from {{ ref('stg_dvf_mutations') }}
16      where nature_mutation = 'Vente'
17  ),
18
19  -- Un même local peut apparaître sur plusieurs lignes (plusieurs parcelles) : on dédouble.
20  locaux as (
21      select distinct
22          id_mutation, type_local, surface_reelle_bati, nombre_pieces_principales, lot1_numero
23      from lignes
24      where type_local is not null
25  ),
26
27  profil_mutation as (
28      select
29          id_mutation,
30          count(*) filter (where type_local in ('Maison', 'Appartement')) as nb_logements
31      ,
32          count(*) filter (where type_local = 'Local industriel. commercial ou assimilé') as nb_loca
33      ux_activite
34      from locaux
35      group by id_mutation
36  ),
37
38  mutation as (
39      -- valeur foncière, date et commune sont identiques sur toutes les lignes d'une mutation
40      select
41          id_mutation,
42          any_value(date_mutation) as date_mutation,
43          any_value(annee) as annee,
44          any_value(valeur_fonciere) as valeur_fonciere,
45          any_value(code_commune) as code_commune,
46          any_value(nom_commune) as nom_commune,
47          any_value(code_departement) as code_departement,
48          any_value(longitude) as longitude,
49          any_value(latitude) as latitude
50      from lignes
51      group by id_mutation
52  ),
53
54  logement as (
55      select
56          id_mutation,
57          any_value(type_local) as type_local,
58          any_value(surface_reelle_bati) as surface_reelle_bati,
59          any_value(nombre_pieces_principales) as nombre_pieces_principales
60      from locaux
61      where type_local in ('Maison', 'Appartement')
62      group by id_mutation
63  )
```

```

62 |
63 | select
64 |     m.id_mutation,
65 |     m.date_mutation,
66 |     m.annee,
67 |     m.code_commune,
68 |     m.nom_commune,
69 |     m.code_departement,
70 |     l.type_local,
71 |     l.surface_reelle_bati,
72 |     l.nombre_pieces_principales,
73 |     m.valeur_fonciere,
74 |     round(m.valeur_fonciere / nullif(l.surface_reelle_bati, 0), 2) as prix_m2,
75 |     m.longitude,
76 |     m.latitude,
77 |     (
78 |         l.surface_reelle_bati >= 9
79 |         and m.valeur_fonciere >= 1000
80 |         and m.valeur_fonciere / l.surface_reelle_bati between 200 and 30000
81 |     ) as est_prix_plausible
82 | from mutation m
83 | join profil_mutation p using (id_mutation)
84 | join logement l using (id_mutation)
85 | where p.nb_logements = 1
86 |     and p.nb_locaux_activite = 0
87 |     -- Surface absente dans DVF (quelques dizaines de cas par an) : prix au m² incalculable.
88 |     and l.surface_reelle_bati > 0

```

Lignes	Ce qui se passe
14-17	Ne garde que la nature « Vente » : ni ventes sur plan (VEFA), ni enchères, ni échanges, dont les prix ne sont pas comparables.
20-25	Liste les locaux distincts de chaque vente. Un même local peut apparaître sur plusieurs lignes quand il s'étend sur plusieurs parcelles : distinct les dédouble.
27-34	Profil de chaque vente : combien de logements, combien de locaux d'activité. count(*) filter (where ...) compte conditionnellement dans un seul passage.
36-50	Une ligne par vente. any_value prend n'importe quelle valeur du groupe : c'est sans risque ici puisque prix, date et commune sont identiques sur toutes les lignes d'une vente.
74	nullif(x, 0) renvoie NULL si la surface vaut 0, ce qui évite une division par zéro.
77-81	Drapeau de plausibilité : surface d'au moins 9 m², prix d'au moins 1 000 €, prix au m² entre 200 et 30 000 €. On marque au lieu de supprimer : on peut compter ce qui est écarté.
85-88	Filtre final : exactement un logement, aucun local d'activité, surface renseignée.

Piège rencontré

Au premier **dbt build** sur les vraies données, le test **not_null** sur **surface_reelle_bati** a échoué : 78 ventes de logement n'avaient pas de surface dans DVF. Les 7 modèles et tests en aval ont été sautés (SKIP), dbt refusant de construire sur une base douteuse. Correction : ligne 88, on écarte ces ventes, qui ne permettent aucun prix au m². Le test a joué son rôle.

int_dpe_logement_dernier : un logement, un DPE

dbt/models/intermediate/int_dpe_logement_dernier.sql

```

1 | /*
2 |     Un logement = son DPE le plus récent.
3 |
4 |     Le DPE n'a pas d'identifiant de logement : un même logement rediagnostiqué
5 |     (vente puis location, correction...) apparaît plusieurs fois. Dans l'Aisne,
6 |     ~30 % des DPE sont ainsi des doublons. On reconstruit une clé logement :
7 |     adresse BAN + type + surface arrondie
8 |     (plusieurs appartements partagent la même adresse ; la surface les distingue).

```

```

9 |
10 | Les DPE « immeuble » (bâtiment entier) sont exclus : ils décrivent un ensemble
11 | de logements et fausseraient les parts calculées par logement.
12 |
13 | Limite : seuls les logements diagnostiqués depuis juillet 2021 (vente, location)
14 | sont présents – c'est un échantillon du parc, pas le parc entier.
15 | */
16 | with dpe as (
17 |     select *
18 |     from {{ ref('stg_dpe_logements') }}
19 |     where type_batiment in ('maison', 'appartement')
20 |     and code_commune is not null
21 | ),
22 |
23 | avec_cle as (
24 |     select
25 |         *,
26 |         coalesce(identifiant_ban, 'sans-adresse-' || numero_dpe)
27 |         || ' ' || type_batiment
28 |         || ' ' || coalesce(cast(round(surface_habitable) as varchar), '?') as cle_logement
29 |     from dpe
30 | )
31 |
32 | select
33 |     cle_logement,
34 |     numero_dpe,
35 |     date_etablissement_dpe,
36 |     code_commune,
37 |     nom_commune,
38 |     code_departement,
39 |     type_batiment,
40 |     periode_construction,
41 |     annee_construction,
42 |     surface_habitable,
43 |     etiquette_dpe,
44 |     etiquette_ges,
45 |     conso_energie_primaire_m2,
46 |     emission_ges_m2,
47 |     etiquette_dpe in ('F', 'G') as est_passoire_thermique,
48 |     etiquette_dpe in ('A', 'B') as est_logement_performant
49 | from avec_cle
50 | qualify row_number() over (
51 |     partition by cle_logement
52 |     order by date_etablissement_dpe desc, numero_dpe desc
53 | ) = 1

```

Le DPE n'a pas d'identifiant de logement : un logement vendu puis loué a deux DPE. Dans l'Aisne, environ 30 % des DPE sont des rediagnostics. On reconstruit une **clé logement** : adresse BAN + type + surface arrondie (plusieurs appartements partagent une adresse, la surface les distingue). Puis `qualify row_number() ... order by date desc = 1` garde le DPE le plus récent.

- Les DPE « immeuble » décrivent un bâtiment entier : on les exclut pour ne pas fausser les parts calculées par logement.
- `coalesce(identifiant_ban, 'sans-adresse-' || numero_dpe)` : un DPE sans adresse devient sa propre clé, plutôt que d'être fusionné avec tous les autres DPE sans adresse.
- Deux indicateurs booléens préparent les calculs : passoire (F ou G) et logement performant (A ou B).

int_insee_commune : du format long au format large

dbt/models/intermediate/int_insee_commune.sql

```

1 | /*
2 | Pivot des indicateurs INSEE : une ligne par commune, une colonne par indicateur,
3 | en gardant pour chaque indicateur sa période la plus récente.
4 |
5 | Millésimes dans l'édition 2026 : population et logements 2023 (recensement),
6 | niveau de vie et pauvreté 2023 (Filosophi).
7 | */
8 | with dernier as (

```

```

9 | select *
10 | from {{ ref('stg_insee_indicateurs') }}
11 | qualify row_number() over (partition by code_commune, indicateur order by annee desc) = 1
12 | )
13 |
14 | select
15 |     code_commune,
16 |     any_value(nom_commune) as nom_commune,
17 |     left(code_commune, 2) as code_departement,
18 |     max(valeur) filter (where indicateur = 'POP') as population,
19 |     max(valeur) filter (where indicateur = 'SUP') as superficie_km2,
20 |     max(valeur) filter (where indicateur = 'MED_SL') as niveau_vie_median
21 | ,
22 |     max(valeur) filter (where indicateur = 'PR_MD60') as taux_pauvrete,
23 |     max(valeur) filter (where indicateur = 'DWELLINGS') as nb_logements,
24 |     max(valeur) filter (where indicateur = 'DWELLINGS_OCS_DW_MAIN') as nb_residences_principales,
25 |     max(valeur) filter (where indicateur = 'DWELLINGS_OCS_DW_MAIN_TSH_100') as nb_proprietaires,
26 |     max(valeur) filter (where indicateur = 'DWELLINGS_OCS_DW_VAC') as nb_logements_vacants,
27 |     max(valeur) filter (where indicateur = 'DWELLINGS_OCS_DW_SEC_DW_OCC') as nb_residences_secondaires,
28 |     bool_or(est_secret_statistique and indicateur = 'MED_SL') as niveau_vie_sous_secret
29 | from dernier
30 | group by code_commune

```

On **pivote** : une ligne par commune et une colonne par indicateur. **max(valeur) filter (where indicateur = 'POP')** récupère la valeur de l'indicateur voulu dans le groupe de la commune (le **max** ne sert qu'à produire une seule valeur, il n'y en a qu'une). Le **qualify** en tête ne garde, pour chaque indicateur, que l'année la plus récente.

dbt/models/intermediate/_intermediate.yml

```

1 | version: 2
2 |
3 | models:
4 |   - name: int_ventes_logement
5 |     description: >
6 |       Ventes d'un logement unique (maison ou appartement), une ligne par mutation,
7 |       avec prix au m² et indicateur de plausibilité.
8 |     columns:
9 |       - name: id_mutation
10 |         tests: [unique, not_null]
11 |       - name: type_local
12 |         tests:
13 |           - accepted_values:
14 |             arguments:
15 |               values: ["Maison", "Appartement"]
16 |       - name: surface_reelle_bati
17 |         tests: [not_null]
18 |       - name: code_commune
19 |         tests:
20 |           # Les codes DVF doivent exister dans le référentiel INSEE (géographie 2026).
21 |           # Alerte seulement : une fusion de communes récente peut laisser un ancien code.
22 |           - relationships:
23 |             arguments:
24 |               to: ref('int_insee_commune')
25 |               field: code_commune
26 |             config:
27 |               severity: warn
28 |
29 |   - name: int_dpe_logement_dernier
30 |     description: Un logement (maison ou appartement) = son DPE le plus récent.
31 |     columns:
32 |       - name: cle_logement
33 |         tests: [unique, not_null]
34 |       - name: code_commune
35 |         tests:
36 |           - relationships:
37 |             arguments:
38 |               to: ref('int_insee_commune')
39 |               field: code_commune

```

```

40 |         config:
41 |             severity: warn
42 |
43 |     - name: int_insee_commune
44 |       description: Indicateurs INSEE pivotés, une ligne par commune.
45 |       columns:
46 |         - name: code_commune
47 |           tests: [unique, not_null]
48 |         - name: population
49 |           tests:
50 |             # Mayotte (976) n'a pas de population 2023 dans l'édition 2026 : hors périmètre DVF de t
           oute façon.
51 |           - not_null:
52 |               config:
53 |                 where: "code_commune not like '976%'"

```

Le test **relationships** vérifie que chaque code commune de DVF et des DPE existe dans le référentiel INSEE (l'équivalent d'une clé étrangère). Il est en alerte parce que quelques communes ont fusionné depuis 2021 : DVF garde leur ancien code (216 ventes et 45 logements sur toute la région). Le test de population exclut Mayotte (**where: "code_commune not like '976%'"**), sans population 2023 dans cette édition et hors du champ de DVF.

4.6 Marts analytiques

dbt/models/marts/mart_prix_commune_annee.sql

```

1 | /*
2 |   Prix immobiliers par commune, année et type de logement.
3 |   Médiane plutôt que moyenne : robuste aux ventes atypiques restantes.
4 |   Les cellules avec moins de 5 ventes sont gardées mais signalées (`est_fiable`),
5 |   pour que le dashboard puisse les griser au lieu d'afficher un chiffre trompeur.
6 | */
7 | with ventes as (
8 |     select * from {{ ref('int_ventes_logement') }}
9 |     where est_prix_plausible
10 | )
11 |
12 | select
13 |     code_commune || '-' || annee || '-' || type_local          as cle,
14 |     code_commune,
15 |     any_value(nom_commune)                                     as nom_commune,
16 |     any_value(code_departement)                               as code_departement,
17 |     annee,
18 |     type_local,
19 |     count(*)                                                   as nb_ventes,
20 |     round(median(prix_m2), 0)                                   as prix_m2_median,
21 |     round(quantile_cont(prix_m2, 0.25), 0)                   as prix_m2_q1,
22 |     round(quantile_cont(prix_m2, 0.75), 0)                   as prix_m2_q3,
23 |     round(median(valeur_fonciere), 0)                         as prix_median,
24 |     round(median(surface_reelle_bati), 1)                    as surface_mediane,
25 |     count(*) >= 5                                              as est_fiable
26 | from ventes
27 | group by code_commune, annee, type_local

```

Prix au m² par commune × année × type. **Médiane plutôt que moyenne** : une villa à 3 M€ fait exploser la moyenne d'un village, pas sa médiane. Les quartiles Q1 et Q3 montrent la dispersion. **est_fiable** signale les cellules de moins de 5 ventes, qu'un dashboard doit griser. La colonne **cle** concatène commune, année et type pour tester l'unicité du grain.

dbt/models/marts/mart_logement_commune.sql

```

1 | /*
2 |   Table centrale du dashboard : une ligne par commune du périmètre.
3 |
4 |   Croise trois sources par le code commune INSEE :
5 |   - INSEE : population, niveau de vie, structure du parc de logements ;
6 |   - DVF   : prix de l'immobilier sur les années récentes (var `annees_recentes`) ;

```

```

7 | - DPE : performance énergétique des logements diagnostiqués.
8 |
9 | Indicateur phare – `annees_niveau_vie_pour_acheter` :
10 | prix médian d'un logement / niveau de vie médian annuel de la commune.
11 | Le niveau de vie est un revenu par « unité de consommation » : l'indicateur
12 | sert à comparer les communes entre elles, pas à mesurer un effort réel de ménage.
13 | */
14 | {% set annees = var('annees_recentes') %}
15 |
16 | with communes as (
17 |     select *
18 |     from {{ ref('int_insee_commune') }}
19 |     where code_departement in ({{ liste_departements() }})
20 | ),
21 |
22 | ventes as (
23 |     select
24 |         code_commune,
25 |         count(*)                                as nb_ventes,
26 |         count(*) filter (where type_local = 'Appartement') as nb_ventes_appa
27 |         rtement,
28 |         count(*) filter (where type_local = 'Maison')      as nb_ventes_mais
29 |         on,
30 |         round(median(prix_m2) filter (where type_local = 'Appartement'), 0) as prix_m2_median
31 |         _appartement,
32 |         round(median(prix_m2) filter (where type_local = 'Maison'), 0)      as prix_m2_median
33 |         _maison,
34 |         round(median(valeur_fonciere), 0)                                as prix_median_lo
35 |         gement,
36 |         round(median(surface_reelle_bati), 1)                            as surface_median
37 |         e_vendue
38 |     from {{ ref('int_ventes_logement') }}
39 |     where est_prix_plausible
40 |         and annee in ({{ annees | join(', ') }})
41 |     group by code_commune
42 | ),
43 |
44 | dpe as (
45 |     select
46 |         code_commune,
47 |         count(*)                                as nb_logements_diagnostiques,
48 |         avg(est_passoire_thermique::int)        as part_passoires_thermiques,
49 |         avg(est_logement_performant::int)       as part_logements_performants,
50 |         round(median(conso_energie_primaire_m2), 0) as conso_energie_mediane_m2,
51 |         mode(etiquette_dpe)                    as etiquette_dpe_la_plus_frequ
52 |     ente
53 |     from {{ ref('int_dpe_logement_dernier') }}
54 |     group by code_commune
55 | )
56 |
57 | select
58 |     c.code_commune,
59 |     c.nom_commune,
60 |     c.code_departement,
61 |
62 |     -- Démographie & revenus (INSEE)
63 |     c.population,
64 |     c.superficie_km2,
65 |     round(c.population / nullif(c.superficie_km2, 0), 1) as densite_hab_km
66 |     2,
67 |     c.niveau_vie_median,
68 |     c.niveau_vie_sous_secret,
69 |     c.taux_pauvrete,
70 |
71 |     -- Parc de logements (INSEE)
72 |     c.nb_logements,
73 |     round(c.nb_logements_vacants / nullif(c.nb_logements, 0), 4) as part_logements
74 |     _vacants,
75 |     round(c.nb_residences_secondaires / nullif(c.nb_logements, 0), 4) as part_residence
76 |     s_secondaires,
77 |     round(c.nb_proprietaires / nullif(c.nb_residences_principales, 0), 4) as part_proprieta
78 |     ires,
79 |
80 |     -- Marché immobilier (DVF, années récentes)
81 |     coalesce(v.nb_ventes, 0) as nb_ventes,
82 |     v.nb_ventes_appartement,

```

```

72 |     v.nb_ventes_maison,
73 |     v.prix_m2_median_appartement,
74 |     v.prix_m2_median_maison,
75 |     v.prix_median_logement,
76 |     v.surface_mediane_vendue,
77 |     coalesce(v.nb_ventes, 0) >= 10                                as prix_est_fiabl
78 | e,
79 |     -- Accessibilité
80 |     round(v.prix_median_logement / nullif(c.niveau_vie_median, 0), 1)    as annees_niveau_
    vie_pour_acheter,
81 |
82 |     -- Performance énergétique (DPE)
83 |     coalesce(d.nb_logements_diagnostiques, 0)                        as nb_logements_d
    iagnostiques,
84 |     round(d.part_passoires_thermiques, 4)                            as part_passoires
    _thermiques,
85 |     round(d.part_logements_performants, 4)                          as part_logements
    _performants,
86 |     d.conso_energie_mediane_m2,
87 |     d.etiquette_dpe_la_plus_frequente,
88 |     coalesce(d.nb_logements_diagnostiques, 0) >= 20                as dpe_est_fiable
89 | ,
90 |     '{{ annees | first }}-{{ annees | last }}'                      as periode_dvf
91 | from communes c
92 | left join ventes v using (code_commune)
93 | left join dpe d using (code_commune)

```

- `{% set annees = var('annees_recentes') %}` lit une variable du projet ; `{{ annees | join(', ') }}` l'écrit dans le SQL sous la forme `2024, 2025`.
- Trois sous-requêtes (INSEE, ventes, DPE) jointes à gauche (`left join`) sur la liste des communes : une commune sans vente reste présente, avec des valeurs vides.
- `avg(est_passoire_thermique::int)` : la moyenne d'un booléen converti en 0/1 donne directement une proportion.
- Indicateur phare : `annees_niveau_vie_pour_acheter` = prix médian / niveau de vie médian. Le niveau de vie INSEE est calculé par unité de consommation : l'indicateur compare les communes, il ne mesure pas l'effort d'un ménage réel.

dbt/models/marts/mart_dpe_commune_etiquette.sql

```

1 | /*
2 |  Répartition des étiquettes DPE par commune et type de logement.
3 |  Sert aux barres empilées A-G du dashboard (et au filtre par période de construction).
4 | */
5 | select
6 |     code_commune || '-' || type_batiment || '-' || etiquette_dpe || '-' || coalesce(periode_constr
    uction, 'inconnue') as cle,
7 |     code_commune,
8 |     any_value(nom_commune)                as nom_commune,
9 |     any_value(code_departement)           as code_departement,
10 |     type_batiment,
11 |     coalesce(periode_construction, 'inconnue') as periode_construction,
12 |     etiquette_dpe,
13 |     count(*)                             as nb_logements
14 | from {{ ref('int_dpe_logement_dernier') }}
15 | group by code_commune, type_batiment, coalesce(periode_construction, 'inconnue'), etiquette_dpe

```

dbt/models/marts/_marts.yml

```

1 | version: 2
2 |
3 | models:
4 |   - name: mart_prix_commune_annee
5 |     description: Prix au m² médian et volumes de ventes par commune × année × type de logement.
6 |     columns:
7 |       - name: cle
8 |         description: Clé technique commune-année-type.

```

```

9 |         tests: [unique, not_null]
10 |     - name: code_commune
11 |       tests: [not_null]
12 |     - name: nb_ventes
13 |       tests: [not_null]
14 |     - name: prix_m2_median
15 |       tests: [not_null]
16 |
17 | - name: mart_logement_commune
18 |   description: >
19 |     Une ligne par commune du périmètre : population, niveau de vie, parc de logements (INSEE),
20 |     prix récents (DVF), performance énergétique (DPE) et indicateur d'accessibilité.
21 |   columns:
22 |     - name: code_commune
23 |       tests: [unique, not_null]
24 |     - name: population
25 |       tests: [not_null]
26 |     - name: annees_niveau_vie_pour_acheter
27 |       description: Prix médian d'un logement / niveau de vie médian annuel de la commune.
28 |     - name: part_passoires_thermiques
29 |       description: Part des logements diagnostiqués classés F ou G.
30 |
31 | - name: mart_dpe_commune_etiquette
32 |   description: Nombre de logements par commune × type × période de construction × étiquette DPE.
33 |   columns:
34 |     - name: cle
35 |       tests: [unique, not_null]
36 |     - name: etiquette_dpe
37 |       tests:
38 |         - accepted_values:
39 |           arguments:
40 |             values: ["A", "B", "C", "D", "E", "F", "G"]

```

4.7 Seed : un référentiel versionné

dbt/seeds/departements.csv

```

1 | code_departement,nom_departement,code_region,nom_region
2 | 02,Aisne,32,Hauts-de-France
3 | 59,Nord,32,Hauts-de-France
4 | 60,Oise,32,Hauts-de-France
5 | 62,Pas-de-Calais,32,Hauts-de-France
6 | 80,Somme,32,Hauts-de-France

```

dbt/seeds/_seeds.yml

```

1 | version: 2
2 |
3 | seeds:
4 |   - name: departements
5 |     description: Référentiel des départements du périmètre (à compléter pour étendre au national).
6 |     config:
7 |       column_types:
8 |         code_departement: varchar
9 |         code_region: varchar
10 |     columns:
11 |       - name: code_departement
12 |         tests: [unique, not_null]

```

Un **seed** est un petit CSV versionné avec le code, que **dbt seed** (ou **dbt build**) charge comme une table. Idéal pour un référentiel de quelques lignes qui ne vient d'aucune source. **column_types** force le code en texte, sinon « 02 » serait lu comme le nombre 2.

4.8 Le schéma en étoile pour Power BI

Les marts précédents sont déjà agrégés. Pour qu'un rapport puisse filtrer librement (par mois, par commune, par type), il faut lui donner la donnée au grain fin, organisée en **schéma en étoile** : des tables de faits (les événements mesurés) entourées de dimensions (les axes d'analyse).

Table	Type	Grain	Lignes (région)
fct_ventes	Fait	1 vente	353 702
fct_dpe_mensuel	Fait	commune × mois × type × période × étiquette	442 474
dim_commune	Dimension	1 commune	3 792
dim_date	Dimension	1 jour, de 2021 à fin 2026	2 191

La raison principale : une médiane ne s'additionne pas

La médiane de Lille et celle de Roubaix ne donnent pas la médiane des deux villes. Si Power BI ne recevait que des médianes précalculées, un filtre « Nord, 2023-2024 » afficherait un résultat faux. En exportant chaque vente, la mesure DAX **MEDIAN()** recalcule la vraie médiane pour n'importe quel filtre. À l'inverse, les DPE sont des **comptages**, qui s'additionnent : on peut les agréger au mois sans rien perdre, ce qui divise le volume par trois.

dbt/models/marts/dim_date.sql

```

1  /*
2   Table de dates (une ligne par jour) pour l'intelligence temporelle Power BI.
3   Couvre du 1er janvier 2021 au 31 décembre de l'année en cours.
4   À marquer comme « table de dates » dans Power BI (colonne `date`).
5  */
6  with jours as (
7      select cast(d as date) as date
8          from generate_series(
9              date '2021-01-01',
10             cast(date_trunc('year', current_date) + interval 1 year - interval 1 day as date),
11             interval 1 day
12          ) as t(d)
13  ),
14  libelles as (
15      select
16          ['janvier', 'février', 'mars', 'avril', 'mai', 'juin', 'juillet',
17           'août', 'septembre', 'octobre', 'novembre', 'décembre'] as mois_fr,
18          ['janv.', 'févr.', 'mars', 'avr.', 'mai', 'juin', 'juil.',
19           'août', 'sept.', 'oct.', 'nov.', 'déc.'] as mois_courts_fr
20  ),
21  ['lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi', 'samedi', 'dimanche'] as jours_fr
22  )
23
24  select
25      date,
26      year(date) as annee,
27      case when month(date) <= 6 then 1 else 2 end as semestre,
28      year(date) || '-S' || case when month(date) <= 6 then 1 else 2 end as annee_semestre,
29      quarter(date) as trimestre,
30      year(date) || '-T' || quarter(date) as annee_trimestre,
31      month(date) as mois,
32      mois_fr[month(date)] as nom_mois,
33      mois_courts_fr[month(date)] as nom_mois_court,
34      strftime(date, '%Y-%m') as annee_mois,
35      -- clé de tri pour afficher « janv. 2024 » dans l'ordre chronologique
36      year(date) * 100 + month(date) as annee_mois_tri,
37      cast(date_trunc('month', date) as date) as premier_jour_mois,
38      isodow(date) as jour_semaine,
39      jours_fr[isodow(date)] as nom_jour,
40      isodow(date) >= 6 as est_weekend
41  from jours, libelles

```

- `generate_series` produit une ligne par jour entre deux dates ; la borne haute se cale sur la fin de l'année en cours.
- Les noms de mois en français viennent de listes : `mois_fr[month(date)]` (les listes DuckDB commencent à l'indice 1).
- `annee_mois_tri` (202401, 202402...) sert à trier chronologiquement la colonne texte `annee_mois` dans Power BI.

dbt/models/marts/dim_commune.sql

```

1  /*
2   Dimension commune : une ligne par commune du périmètre.
3
4   - Attributs INSEE (population, niveau de vie, parc de logements) : photo 2023.
5   - Coordonnées : point central calculé à partir des ventes DVF géolocalisées
6     (médiane des latitudes / longitudes), pour placer les communes sur une carte.
7   - Communes « historiques » : codes présents dans DVF ou les DPE mais disparus du
8     référentiel INSEE 2026 (fusions de communes). On les garde pour que les faits
9     restent rattachés à une commune, avec `est_commune_historique = true`.
10  */
11  with insee as (
12      select *
13      from {{ ref('int_insee_commune') }}
14      where code_departement in ({{ liste_departements() }})
15  ),
16
17  coordonnees as (
18      select
19          code_commune,
20          round(median(latitude), 5) as latitude,
21          round(median(longitude), 5) as longitude
22      from {{ ref('stg_dvf_mutations') }}
23      where latitude is not null and longitude is not null
24      group by code_commune
25  ),
26
27  codes_faits as (
28      select code_commune, nom_commune, code_departement from {{ ref('stg_dvf_mutations') }}
29      union all
30      select code_commune, nom_commune, code_departement from {{ ref('stg_dpe_logements') }}
31  ),
32
33  historiques as (
34      select
35          code_commune,
36          any_value(nom_commune) as nom_commune,
37          any_value(code_departement) as code_departement
38      from codes_faits
39      where code_departement in ({{ liste_departements() }})
40      and code_commune not in (select code_commune from insee)
41      group by code_commune
42  ),
43
44  communes as (
45      select code_commune, nom_commune, code_departement, false as est_commune_historique from insee
46      union all
47      select code_commune, nom_commune, code_departement, true from historiques
48  )
49
50  select
51      c.code_commune,
52      c.nom_commune,
53      c.nom_commune || ' (' || c.code_departement || ')' as nom_commune_dep,
54      c.code_departement,
55      d.nom_departement,
56      d.nom_region,
57      c.est_commune_historique,
58      xy.latitude,
59      xy.longitude,
60      i.population,
61      i.superficie_km2,
62      round(i.population / nullif(i.superficie_km2, 0), 1) as densite_hab_km2,
63      case

```

```

64 |         when i.population is null then null
65 |         when i.population < 2000 then '1. Moins de 2 000 hab.'
66 |         when i.population < 10000 then '2. 2 000 à 10 000 hab.'
67 |         when i.population < 50000 then '3. 10 000 à 50 000 hab.'
68 |         else '4. Plus de 50 000 hab.'
69 |     end                                                                 as taille_commune,
70 |     i.niveau_vie_median,
71 |     i.niveau_vie_sous_secret,
72 |     i.taux_pauvrete,
73 |     -- effectifs INSEE pondérés (décimaux dans la source) : arrondis
74 |     round(i.nb_logements)::integer as nb_logements,
75 |     round(i.nb_residences_principales)::integer as nb_residences_principales
76 | ,
77 |     round(i.nb_logements_vacants)::integer as nb_logements_vacants,
78 |     round(i.nb_residences_secondaires)::integer as nb_residences_secondaires
79 | ,
80 |     round(i.nb_proprietaires)::integer as nb_proprietaires
81 | from communes c
82 | left join {{ ref('departements') }} d using (code_departement)
83 | left join coordonnees xy using (code_commune)
84 | left join {{ ref('int_insee_commune') }} i using (code_commune)

```

- **Coordonnées** : la médiane des latitudes et longitudes de toutes les ventes géolocalisées de la commune donne un point central, sans nouvelle source.
- **Communes historiques** : les codes présents dans les faits mais absents du référentiel INSEE (fusions) sont ajoutés avec `est_commune_historique = true`. Sans eux, leurs ventes seraient orphelines dans Power BI.
- La tranche de taille est préfixée « 1. », « 2. »... pour que l'ordre alphabétique soit aussi l'ordre logique.
- Les effectifs INSEE sont pondérés (13 539,56 logements) : on les arrondit à l'entier.

dbt/models/marts/fct_ventes.sql

```

1 | /*
2 |   Fait « vente » : une ligne par vente d'un logement, au grain le plus fin.
3 |
4 |   On exporte le détail (et non des médianes précalculées) car une médiane ne
5 |   s'additionne pas : Power BI la recalcule avec MEDIANX selon les filtres actifs
6 |   (période, commune, type...). Seules les ventes au prix plausible sont gardées.
7 |
8 |   Relations Power BI :
9 |     fct_ventes[date_vente] → dim_date[date]
10 |    fct_ventes[code_commune] → dim_commune[code_commune]
11 | */
12 | select
13 |     id_mutation,
14 |     date_mutation as date_vente,
15 |     code_commune,
16 |     type_local as type_logement,
17 |     surface_reelle_bati as surface_m2,
18 |     nombre_pieces_principales as nb_pieces,
19 |     valeur_fonciere as prix,
20 |     prix_m2,
21 |     latitude,
22 |     longitude
23 | from {{ ref('int_ventes_logement') }}
24 | where est_prix_plausible
25 | and code_departement in ({{ liste_departements() }})

```

Une ligne par vente plausible du périmètre, avec des noms de colonnes lisibles dans Power BI (`date_vente`, `prix`, `surface_m2`).

dbt/models/marts/fct_dpe_mensuel.sql

```
1  /*
2  Fait « DPE réalisés » agrégé au mois : commune × mois × type × période de construction × étiquet
   te.
3
4  Ce sont des flux (diagnostics réalisés dans le mois), pas le parc : un logement
5  rediagnostiqué compte à chaque diagnostic. Pour une photo du parc, voir
6  mart_logement_commune (un logement = son dernier DPE).
7
8  Les comptages sont additifs : la part de passoires (F+G / total) reste juste
9  quel que soit le filtre appliqué dans Power BI.
10
11  Relations Power BI :
12      fct_dpe_mensuel[mois]          → dim_date[date]      (1er jour du mois)
13      fct_dpe_mensuel[code_commune] → dim_commune[code_commune]
14  */
15  with dpe as (
16      select *
17      from {{ ref('stg_dpe_logements') }}
18      where type_batiment in ('maison', 'appartement')
19            and code_departement in ({{ liste_departements() }})
20            and date_etablissement_dpe is not null
21  )
22
23  select
24      cast(date_trunc('month', date_etablissement_dpe) as date)          as mois,
25      code_commune,
26      type_batiment                                                    as type_logement,
27      coalesce(periode_construction, 'inconnue')                      as periode_construction,
28      -- ordre chronologique des périodes (« Trier par colonne » dans Power BI)
29      case periode_construction
30          when 'avant 1948' then 1 when '1948-1974' then 2 when '1975-1977' then 3
31          when '1978-1982' then 4 when '1983-1988' then 5 when '1989-2000' then 6
32          when '2001-2005' then 7 when '2006-2012' then 8 when '2013-2021' then 9
33          when 'après 2021' then 10 else 99
34      end                                                                as periode_construction_tri
35  ,
36      etiquette_dpe,
37      etiquette_dpe in ('F', 'G')                                      as est_passoire_thermique,
38      count(*)                                                         as nb_dpe,
39      round(sum(surface_habitable), 1)                                as surface_totale_m2
40  from dpe
41  group by all
```

- Ici, on compte les DPE **réalisés** (un flux), pas le parc : un logement rediagnostiqué compte à chaque diagnostic. C'est ce qu'il faut pour suivre l'évolution mois par mois.
- **group by all** (DuckDB) regroupe automatiquement par toutes les colonnes non agrégées.
- **periode_construction_tri** donne l'ordre chronologique des périodes, que l'ordre alphabétique mélangerait (« après 2021 » avant « avant 1948 »).

dbt/models/marts/_star.yml

```
1  version: 2
2
3  # Schéma en étoile exporté vers Power BI.
4  models:
5      - name: dim_date
6        description: Une ligne par jour depuis 2021, libellés en français. Table de dates Power BI.
7        columns:
8            - name: date
9              tests: [unique, not_null]
10
11      - name: dim_commune
12        description: >
13          Une ligne par commune du périmètre : attributs INSEE (2023), département, coordonnées
14          (médiane des ventes DVF géolocalisées). Inclut les communes fusionnées encore présentes
15          dans les faits ('est_commune_historique').
16        columns:
17            - name: code_commune
18              tests: [unique, not_null]
19            - name: nom_departement
```

```

20 |         tests: [not_null]
21 |
22 |     - name: fct_ventes
23 |       description: Une ligne par vente d'un logement (prix plausible). Médianes à calculer en DAX (M
24 |       columns:
25 |         - name: id_mutation
26 |           tests: [unique, not_null]
27 |         - name: date_vente
28 |           tests:
29 |             - not_null
30 |             - relationships:
31 |               arguments:
32 |                 to: ref('dim_date')
33 |                 field: date
34 |         - name: code_commune
35 |           tests:
36 |             - not_null
37 |             - relationships:
38 |               arguments:
39 |                 to: ref('dim_commune')
40 |                 field: code_commune
41 |
42 |     - name: fct_dpe_mensuel
43 |       description: DPE réalisés par commune x mois x type x période de construction x étiquette (flu
44 |       tests:
45 |         - unique_combinaison:
46 |           arguments:
47 |             colonnes: [mois, code_commune, type_logement, periode_construction, etiquette_dpe]
48 |       columns:
49 |         - name: mois
50 |           tests:
51 |             - not_null
52 |             - relationships:
53 |               arguments:
54 |                 to: ref('dim_date')
55 |                 field: date
56 |         - name: code_commune
57 |           tests:
58 |             - relationships:
59 |               arguments:
60 |                 to: ref('dim_commune')
61 |                 field: code_commune
62 |         - name: nb_dpe
63 |           tests: [not_null]

```

Les tests **relationships** sont ici bloquants : chaque fait doit trouver sa date et sa commune, sinon Power BI afficherait une ligne « (vide) ». Le test générique maison vérifie le grain de **fct_dpe_mensuel**.

4.9 Les tests singuliers

Un test singulier est une requête SQL écrite à la main dans **tests/** : elle doit renvoyer zéro ligne.

dbt/tests/assert_nature_mutation_connue.sql

```

1 | -- Échoue si la DGFIP introduit une nature de mutation inconnue (à intégrer aux règles métier).
2 | -- Test singulier plutôt qu'accepted_values : les libellés contiennent des apostrophes.
3 | select nature_mutation, count(*) as nb_lignes
4 | from {{ ref('stg_dvf_mutations') }}
5 | where nature_mutation not in (
6 |   'Vente',
7 |   'Vente en l'état futur d'achèvement',
8 |   'Vente terrain à bâtir',
9 |   'Adjudication',
10 |   'Expropriation',
11 |   'Echange'
12 | )
13 | group by nature_mutation

```

dbt/tests/assert_prix_m2_median_dans_bornes.sql

```
1 | -- Échoue si une médiane sort des bornes de plausibilité : signe d'un filtre cassé en amont.
2 | select *
3 | from {{ ref('mart_prix_commune_annee') }}
4 | where prix_m2_median not between 200 and 30000
```

dbt/tests/assert_parts_entre_0_et_1.sql

```
1 | -- Toutes les parts (ratios) du mart communal doivent être comprises entre 0 et 1.
2 | select code_commune, nom_commune
3 | from {{ ref('mart_logement_commune') }}
4 | where part_logements_vacants      not between 0 and 1
5 |    or part_residences_secondaires not between 0 and 1
6 |    or part_proprietaires          not between 0 and 1
7 |    or part_passoires_thermiques   not between 0 and 1
8 |    or part_logements_performants  not between 0 and 1
```

4.10 Lancer et lire dbt

Terminal, depuis dbt/

```
cd dbt
dbt build --profiles-dir . # seeds + modèles + tests, dans l'ordre des dépendances
dbt build --profiles-dir . -s fct_ventes+ # un modèle et tout ce qui en dépend
dbt build --profiles-dir . --vars '{departements: ["02"]}' # surcharger une variable
dbt docs generate --profiles-dir . && dbt docs serve --profiles-dir . # documentation + lignage
```

Résultat	Signification	Action
PASS	Test réussi	Rien
WARN	Test en alerte (severity: warn)	Lire, décider si c'est attendu
ERROR / FAIL	Modèle en erreur ou test échoué	Lire le message ; target/run_results.json détaille chaque résultat
SKIP	Sauté parce qu'un parent a échoué	Corriger le parent, les enfants suivront

Résultat actuel sur la région : **PASS=68, WARN=2, ERROR=0**. Les deux alertes sont les communes fusionnées.

5. Exporter vers Neon

Power BI tourne dans Windows (Parallels) et ne peut pas lire facilement un fichier DuckDB posé sur le Mac. On copie donc les tables finales dans Neon, la base PostgreSQL cloud que tu utilises déjà pour le projet RH. Power BI s'y connecte comme à n'importe quel serveur Postgres.

5.1 Le secret de connexion : .env

.env.example

```
1 | # Copier ce fichier en .env et remplacer par ta vraie chaîne de connexion Neon
2 | # (console Neon → Connect → copier la « connection string »).
3 | NEON_URL=postgresql://neondb_owner:MOT_DE_PASSE@ep-xxxx.region.aws.neon.tech/neondb?sslmode=require
```

Terminal (Mac)

```
cp .env.example .env      # copie le modèle
open -e .env              # l'ouvre dans TextEdit : coller la vraie chaîne après NEON_URL=
```

- La chaîne de connexion se copie dans la console Neon : bouton **Connect**, base **neondb**, « Show password ».
- **.env** est listé dans **.gitignore** : ton mot de passe ne part jamais sur GitHub. **.env.example**, lui, est versionné et montre le format attendu, sans secret.
- **?sslmode=require** impose une connexion chiffrée, exigée par Neon.
- On a gardé ton projet Neon existant : l'export crée un schéma séparé, **open_data_logement**, à côté du schéma **prod** du projet RH. Les schémas sont des dossiers dans une même base.

5.2 export_neon.py

export/export_neon.py

```
1 | """Exporte les tables finales de DuckDB vers Neon PostgreSQL, pour Power BI.
2 |
3 | Power BI (dans Parallels) lit Neon dans le cloud : plus simple et plus fiable que
4 | de lui faire lire un fichier DuckDB sur le Mac.
5 |
6 | Chaque table est remplacée dans une transaction (DROP + CREATE + COPY + COMMIT) :
7 | si l'export plante, l'ancienne version reste intacte dans Neon.
8 |
9 | Configuration : un fichier '.env' à la racine du projet (jamais versionné) contenant
10 |     NEON_URL=postgresql://neondb_owner:MOT_DE_PASSE@HOTE/neondb?sslmode=require
11 |
12 | Usage :
13 |     python export/export_neon.py
14 |     python export/export_neon.py --tables fct_ventes dim_date
15 | """
16 | import argparse
17 | import os
18 | import sys
19 | import time
20 | from pathlib import Path
21 |
22 | import duckdb
23 | import psycpg
24 |
25 | ROOT = Path(__file__).resolve().parents[1]
26 | WAREHOUSE = ROOT / "data" / "warehouse.duckdb"
27 | SCHEMA_CIBLE = "open_data_logement"
28 |
29 | # Tables exposées à Power BI (schéma DuckDB `marts`)
```

```

30 | TABLES = [
31 |     "dim_date",
32 |     "dim_commune",
33 |     "fct_ventes",
34 |     "fct_dpe_mensuel",
35 |     "mart_logement_commune",
36 |     "mart_dpe_commune_etiquette",
37 |     "mart_prix_commune_annee",
38 | ]
39 |
40 | # Correspondance des types DuckDB → PostgreSQL
41 | TYPES = {
42 |     "VARCHAR": "text",
43 |     "BOOLEAN": "boolean",
44 |     "DATE": "date",
45 |     "TIMESTAMP": "timestamp",
46 |     "TIMESTAMP WITH TIME ZONE": "timestampz",
47 |     "TINYINT": "smallint",
48 |     "SMALLINT": "smallint",
49 |     "INTEGER": "integer",
50 |     "BIGINT": "bigint",
51 |     "HUGEINT": "numeric",
52 |     "UBIGINT": "numeric",
53 |     "DOUBLE": "double precision",
54 |     "FLOAT": "real",
55 | }
56 |
57 |
58 | def type_pg(type_duckdb: str) -> str:
59 |     if type_duckdb.startswith("DECIMAL"):
60 |         return type_duckdb.replace("DECIMAL", "numeric")
61 |     if type_duckdb not in TYPES:
62 |         raise ValueError(f"Type DuckDB non géré : {type_duckdb}")
63 |     return TYPES[type_duckdb]
64 |
65 |
66 | def lire_env() -> str:
67 |     """Lit NEON_URL depuis l'environnement ou le fichier .env."""
68 |     env = ROOT / ".env"
69 |     if env.exists():
70 |         for ligne in env.read_text().splitlines():
71 |             ligne = ligne.strip()
72 |             if ligne and not ligne.startswith("#") and "=" in ligne:
73 |                 cle, valeur = ligne.split("=", 1)
74 |                 os.environ.setdefault(cle.strip(), valeur.strip().strip('\"').strip('\''))
75 |     url = os.environ.get("NEON_URL")
76 |     if not url:
77 |         raise SystemExit("NEON_URL introuvable : crée le fichier .env (voir README).")
78 |     return url
79 |
80 |
81 | def exporter_table(duck: duckdb.DuckDBPyConnection, pg: psycopg.Connection, table: str) -> int:
82 |     colonnes = duck.execute(f"describe marts.{table}").fetchall()
83 |     noms = [c[0] for c in colonnes]
84 |     ddl = ",\n".join(f'"{nom}" {type_pg(typ)}' for nom, typ, *_ in colonnes)
85 |     cible = f'"{SCHEMA_CIBLE}"."{table}"'
86 |
87 |     lignes = duck.execute(f"select * from marts.{table}").fetchall()
88 |     with pg.transaction():
89 |         pg.execute(f"drop table if exists {cible}")
90 |         pg.execute(f"create table {cible} (\n        {ddl}\n)")
91 |         with pg.cursor().copy(f"copy {cible} ({', '.join(chr(34) + n + chr(34) for n in noms)}) f
rom stdin") as copy:
92 |             for ligne in lignes:
93 |                 copy.write_row(ligne)
94 |     return len(lignes)
95 |
96 |
97 | def main() -> int:
98 |     p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpF
ormatter)
99 |     p.add_argument("--tables", nargs="+", default=TABLES, choices=TABLES)
100 |     args = p.parse_args()
101 |
102 |     url = lire_env()
103 |     if not WAREHOUSE.exists():

```

```

104 |         raise SystemExit("warehouse.duckdb introuvable : lance d'abord load_raw.py puis dbt build
105 |         .")
106 |         with duckdb.connect(str(WAREHOUSE), read_only=True) as duck, psycopg.connect(url, autocommit=
107 |         True, prepare_threshold=None) as pg:
108 |             pg.execute(f'create schema if not exists "{SCHEMA_CIBLE}"')
109 |             for table in args.tables:
110 |                 debut = time.time()
111 |                 n = exporter_table(duck, pg, table)
112 |                 print(f"{table:<28} {n:>10,} lignes  ({time.time() - debut:.1f}s)")
113 |             print(f"\nExport terminé vers le schéma « {SCHEMA_CIBLE} » de Neon.")
114 |             return 0
115 |
116 | if __name__ == "__main__":
117 |     sys.exit(main())

```

Partie	Ce qui se passe
TABLES	Les 7 tables exposées à Power BI : les 4 du schéma en étoile et les 3 marts analytiques.
TYPES / type_pg	Traduit chaque type DuckDB en type PostgreSQL (VARCHAR → text, DOUBLE → double precision...). Un type inconnu arrête le script avec un message clair plutôt que de créer une colonne fausse.
lire_env	Lit .env sans bibliothèque supplémentaire. setdefault laisse la priorité à une variable d'environnement déjà définie (utile pour l'automatisation).
exporter_table	Lit la structure (describe) et les lignes dans DuckDB, puis, dans une transaction : supprime l'ancienne table, la recrée, et copie les lignes avec COPY , la méthode d'insertion en masse la plus rapide de PostgreSQL.
main	DuckDB est ouvert en lecture seule ; prepare_threshold=None désactive les requêtes préparées, que le pooler de connexions de Neon (PgBouncer) gère mal.

Pourquoi une transaction

Les quatre opérations (drop, create, copy, commit) forment un bloc indivisible : si l'export plante au milieu, PostgreSQL annule tout et l'ancienne version de la table reste en place. Power BI ne voit jamais une table vide ou à moitié remplie.

Le script a été testé sur un PostgreSQL 16 local avant de t'être envoyé : types, nombre de lignes et médianes recalculées côté Postgres étaient conformes.

Piège rencontré

Après l'export, les tables n'apparaissaient pas dans la console Neon : la vue « Tables » affiche par défaut le schéma **public**. Il suffisait de choisir **open_data_logement** dans le menu « Schema ». En cas de doute, cette requête dans le SQL Editor liste les tables : **select table_name from information_schema.tables where table_schema = 'open_data_logement' ;**

6. Journal des incidents

Un pipeline réel ne marche jamais du premier coup. Ce journal vaut autant que le code : c'est exactement ce qu'on raconte en entretien quand on te demande « une difficulté rencontrée et comment tu l'as traitée ».

Symptôme	Cause	Résolution
dbt : test <code>accepted_values</code> en erreur de syntaxe	Apostrophes dans « Vente en l'état futur d'achèvement »	Test singulier SQL avec apostrophes doublées
dbt : <code>not_null surface_reelle_bati</code> FAIL 78, 7 SKIP	78 ventes DVF sans surface	Exclusion explicite dans <code>int_ventes_logement</code> (pas de prix au m ² possible)
dbt : <code>not_null population</code> FAIL 17	Mayotte sans population 2023 dans l'édition INSEE	Test restreint avec <code>where</code> (hors champ DVF)
dbt : 2 WARN <code>relationships</code>	Communes fusionnées depuis 2021, ancien code gardé dans DVF/DPE	Alerte acceptée et documentée ; communes « historiques » ajoutées à <code>dim_commune</code>
Téléchargement DPE très lent, timeouts	API ADEME saturée par moments	Pages de 5 000, 6 essais, reprise sur incident, <code>caffeinate</code>
Codes INSEE introuvables dans la documentation	Nouveau format « long » de l'INSEE	Téléchargement puis lecture des codes réels avant d'écrire le modèle
Git : « unable to auto-detect email »	Identité Git non configurée	<code>git config --global user.name / user.email</code>
GitHub : « Password authentication is not supported »	GitHub exige un jeton depuis 2021	Jeton classic (repo + workflow) collé comme mot de passe
Homebrew : « no bottles for macOS 13 »	macOS 13 n'est plus supporté par Homebrew	Abandon de GitHub CLI au profit du jeton
<code>pip install</code> : fichier introuvable	Terminal resté dans <code>dbt/</code>	<code>cd ~/Documents/open-data-logement</code>
Tables absentes dans la console Neon	Vue positionnée sur le schéma <code>public</code>	Choisir le schéma <code>open_data_logement</code>

6.1 Un point encore ouvert : les DPE de 2025

Le nombre de DPE réalisés en 2025 baisse fortement dans trois départements alors qu'il monte dans les deux autres :

Département	2024	2025	Évolution
Aisne (02)	20 726	26 089	+26 %
Nord (59)	150 960	114 249	-24 %
Oise (60)	31 954	37 116	+16 %
Pas-de-Calais (62)	89 326	52 601	-41 %
Somme (80)	30 301	19 172	-37 %

Le téléchargement est complet : le total de chaque département correspond exactement à celui annoncé par l'API, sans doublon. L'écart vient donc des données de l'ADEME elles-mêmes, mais il est inhabituel entre

départements voisins. À faire : relancer `download_dpe.py --deps 59 62 80 --force` dans quelques semaines et comparer. En attendant, ne pas en tirer de conclusion dans le dashboard.

7. Préparer Power BI

7.1 Se connecter à Neon

- 1 Power BI Desktop → **Obtenir des données** → **Base de données PostgreSQL**.
- 2 Serveur : `ep-spring-fire-am8oqesr-pooler.c-5.us-east-1.aws.neon.tech` • Base : `neondb` • Mode **Importer**.
- 3 Cocher `open_data_logement.dim_date`, `.dim_commune`, `.fct_ventes`, `.fct_dpe_mensuel`, ainsi que `.mart_logement_commune` et `.mart_dpe_commune_etiquette`.

7.2 Monter le modèle

Réglage	Où	Pourquoi
Relations <code>fct_ventes[date_vente]</code> → <code>dim_date[date]</code> , <code>fct_dpe_mensuel[mois]</code> → <code>dim_date[date]</code>	Vue Modèle	Plusieurs-à-un, filtrage à sens unique : les dimensions filtrent les faits
Relations <code>code_commune</code> des faits et marts → <code>dim_commune[code_commune]</code>	Vue Modèle	Idem
Marquer <code>dim_date</code> comme table de dates	Clic droit sur la table	Indispensable pour SAMEPERIODYLASTYEAR, TOTALYTD, DATESINPERIOD
Trier <code>nom_mois</code> par <code>mois</code> , <code>annee_mois</code> par <code>annee_mois_tri</code> , <code>periode_construction</code> par <code>periode_construction_tri</code>	Outils de colonne → Trier par colonne	Ordre chronologique au lieu d'alphabétique
Catégories <code>latitude</code> / <code>longitude</code>	Outils de colonne	Placement correct sur les cartes
Masquer les clés côté faits	Vue Modèle	On filtre toujours par les dimensions
Enregistrer en <code>.pbip</code> dans <code>dashboard/</code>	Fichier → Enregistrer sous	Rapport et modèle en fichiers texte, versionnables avec Git

7.3 Les mesures DAX

Une **mesure** est un calcul évalué à la volée selon les filtres actifs du rapport (le « contexte de filtre »). Les mesures se rangent dans une table vide `_Mesures`. Le fichier complet :

`dashboard/mesures.dax`

```

1 | // =====
2 | // Mesures DAX – dashboard Open Data Logement
3 | // À créer dans une table de mesures vide nommée « _Mesures »
4 | // (Accueil → Entrer des données → table vide → renommer « _Mesures »).
5 | // Pré-requis : dim_date marquée comme table de dates (colonne `date`).
6 | // =====
7 |
8 |
9 | // -----
10 | // MARCHÉ IMMOBILIER (fct_ventes)
11 | // -----
12 |
13 | Nb ventes =
14 | COUNTROWS ( fct_ventes )

```

```

15 |
16 | // La médiane est recalculée sur les ventes filtrées : juste à tout niveau
17 | // (commune, département, année, mois, type de logement...)
18 | Prix m² médian =
19 | MEDIAN ( fct_ventes[prix_m2] )
20 |
21 | Prix médian =
22 | MEDIAN ( fct_ventes[prix] )
23 |
24 | Surface médiane =
25 | MEDIAN ( fct_ventes[surface_m2] )
26 |
27 |
28 | // --- Comparaison à l'année précédente -----
29 |
30 | Nb ventes N-1 =
31 | CALCULATE ( [Nb ventes], SAMEPERIODLASTYEAR ( dim_date[date] ) )
32 |
33 | Évol. ventes vs N-1 =
34 | DIVIDE ( [Nb ventes] - [Nb ventes N-1], [Nb ventes N-1] )
35 |
36 | Prix m² médian N-1 =
37 | CALCULATE ( [Prix m² médian], SAMEPERIODLASTYEAR ( dim_date[date] ) )
38 |
39 | Évol. prix m² vs N-1 =
40 | DIVIDE ( [Prix m² médian] - [Prix m² médian N-1], [Prix m² médian N-1] )
41 |
42 |
43 | // --- Tendances -----
44 |
45 | // Lisse la saisonnalité (pics de ventes en été)
46 | Nb ventes 12 mois glissants =
47 | CALCULATE (
48 |     [Nb ventes],
49 |     DATESINPERIOD ( dim_date[date], MAX ( dim_date[date] ), -12, MONTH )
50 | )
51 |
52 | Prix m² médian 12 mois glissants =
53 | CALCULATE (
54 |     [Prix m² médian],
55 |     DATESINPERIOD ( dim_date[date], MAX ( dim_date[date] ), -12, MONTH )
56 | )
57 |
58 | Nb ventes cumul annuel =
59 | TOTALYTD ( [Nb ventes], dim_date[date] )
60 |
61 |
62 | // -----
63 | // PERFORMANCE ÉNERGÉTIQUE (fct_dpe_mensuel)
64 | // -----
65 |
66 | Nb DPE =
67 | SUM ( fct_dpe_mensuel[nb_dpe] )
68 |
69 | Nb passoires thermiques =
70 | CALCULATE ( [Nb DPE], fct_dpe_mensuel[est_passoire_thermique] = TRUE () )
71 |
72 | // Additif : juste quel que soit le filtre
73 | Part passoires thermiques =
74 | DIVIDE ( [Nb passoires thermiques], [Nb DPE] )
75 |
76 | Part passoires N-1 =
77 | CALCULATE ( [Part passoires thermiques], SAMEPERIODLASTYEAR ( dim_date[date] ) )
78 |
79 | // En points de pourcentage (ex. -0,05 = -5 points)
80 | Écart passoires vs N-1 (pts) =
81 | [Part passoires thermiques] - [Part passoires N-1]
82 |
83 | Part logements A-B =
84 | DIVIDE (
85 |     CALCULATE ( [Nb DPE], fct_dpe_mensuel[etiquette_dpe] IN { "A", "B" } ),
86 |     [Nb DPE]
87 | )
88 |
89 |
90 | // -----

```

```

91 // ACCESSIBILITÉ (croisement ventes × revenus)
92 // -----
93
94 // Niveau de vie médian de la commune sélectionnée (vide si plusieurs communes :
95 // une médiane de médianes n'a pas de sens)
96 Niveau de vie médian =
97 SELECTEDVALUE ( dim_commune[niveau_vie_median] )
98
99 // Nombre d'années de niveau de vie médian pour acheter le logement médian
100 Années de niveau de vie pour acheter =
101 DIVIDE ( [Prix médian], [Niveau de vie médian] )
102
103
104 // -----
105 // AFFICHAGE
106 // -----
107
108 // Titre dynamique : « Période : 2021 – 2025 »
109 Titre période =
110 "Période : " & YEAR ( MIN ( fct_ventes[date_vente] ) ) & " – " & YEAR ( MAX ( fct_ventes[date_vente] ) )
111
112 Dernière vente connue =
113 FORMAT ( MAX ( fct_ventes[date_vente] ), "d mmmm yyyy" )
114
115
116 // -----
117 // LIBELLÉS DES CARTES (page 1)
118 // Le segment Année n'existe que sur la page 1 : c'est un segment normal
119 // sur dim_date[annee], et les mesures N-1 (SAMEPERIODLASTYEAR) suffisent.
120 // -----
121
122 // « ▲ +12,9 % vs 2024 » : à utiliser en étiquette de référence de la carte
123 // (dans FORMAT, le séparateur décimal s'écrit toujours « . » ; Power BI
124 // affiche ensuite la virgule selon les paramètres régionaux français)
125 Libellé évol. ventes =
126 VAR e = [Évol. ventes vs N-1]
127 RETURN
128 IF (
129     NOT ISBLANK ( e ),
130     IF ( e >= 0, "▲ ", "▼ " ) & FORMAT ( e, "+0.0 %;-0.0 %" ) & " vs " & ( MAX ( dim_date[annee] ) - 1 )
131 )
132
133 Libellé évol. prix m² =
134 VAR e = [Évol. prix m² vs N-1]
135 RETURN
136 IF (
137     NOT ISBLANK ( e ),
138     IF ( e >= 0, "▲ ", "▼ " ) & FORMAT ( e, "+0.0 %;-0.0 %" ) & " vs " & ( MAX ( dim_date[annee] ) - 1 )
139 )
140
141 // Couleur du libellé : vert si hausse, rouge si baisse (fx → Valeur du champ)
142 Couleur évol. ventes =
143 IF ( [Évol. ventes vs N-1] >= 0, "#1D7A3A", "#B3261E" )
144
145 Couleur évol. prix m² =
146 IF ( [Évol. prix m² vs N-1] >= 0, "#1D7A3A", "#B3261E" )
147
148 // Pour les passoires, une baisse est une bonne nouvelle : couleurs inversées
149 Couleur écart passoires =
150 IF ( [Écart passoires vs N-1 (pts)] <= 0, "#1D7A3A", "#B3261E" )
151
152 // Étiquette « -22 % » sous chaque colonne de l'histogramme annuel (page 2)
153 Libellé évol. ventes vs N-1 =
154 FORMAT ( [Évol. ventes vs N-1], "+0 %;-0 %" )
155
156
157 // =====
158 // VARIANTE OPTIONNELLE : SEGMENT ANNÉE DÉCONNECTÉ
159 // À utiliser seulement si tu veux qu'un graphique garde TOUTES les années
160 // tout en mettant l'année choisie en évidence (ex. ventes mensuelles
161 // 2021 → 2025 avec l'année choisie en bleu foncé).
162 //
163 // 1. Modélisation → Nouvelle table :

```

```

164 //      Années = VALUES ( dim_date[annee] )
165 //      Ne crée AUCUNE relation avec cette table.
166 //      2. Le segment Année pointe sur Années[annee] (sélection unique).
167 //      3. Les cartes utilisent les mesures « (année choisie) » ci-dessous.
168 // =====
169
170 Année choisie =
171 SELECTEDVALUE ( Années[annee], 2025 )
172
173 Année précédente =
174 [Année choisie] - 1
175
176
177 // --- Marché : année choisie vs année précédente -----
178
179 Nb ventes (année choisie) =
180 CALCULATE ( [Nb ventes], TREATAS ( { [Année choisie] }, dim_date[annee] ) )
181
182 Nb ventes (année précédente) =
183 CALCULATE ( [Nb ventes], TREATAS ( { [Année précédente] }, dim_date[annee] ) )
184
185 Évol. ventes (année choisie) =
186 DIVIDE ( [Nb ventes (année choisie)] - [Nb ventes (année précédente)], [Nb ventes (année précéden
    te)] )
187
188 Prix m² médian (année choisie) =
189 CALCULATE ( [Prix m² médian], TREATAS ( { [Année choisie] }, dim_date[annee] ) )
190
191 Prix m² médian (année précédente) =
192 CALCULATE ( [Prix m² médian], TREATAS ( { [Année précédente] }, dim_date[annee] ) )
193
194 Évol. prix m² (année choisie) =
195 DIVIDE ( [Prix m² médian (année choisie)] - [Prix m² médian (année précédente)], [Prix m² médian
    (année précédente)] )
196
197 Prix médian (année choisie) =
198 CALCULATE ( [Prix médian], TREATAS ( { [Année choisie] }, dim_date[annee] ) )
199
200
201 // --- Énergie : année choisie -----
202
203 Nb DPE (année choisie) =
204 CALCULATE ( [Nb DPE], TREATAS ( { [Année choisie] }, dim_date[annee] ) )
205
206 Part passoires (année choisie) =
207 CALCULATE ( [Part passoires thermiques], TREATAS ( { [Année choisie] }, dim_date[annee] ) )
208
209 Part passoires (année précédente) =
210 CALCULATE ( [Part passoires thermiques], TREATAS ( { [Année précédente] }, dim_date[annee] ) )
211
212 // En points (ex. -0,05 = -5 points) ; une baisse est une bonne nouvelle
213 Écart passoires (pts) =
214 [Part passoires (année choisie)] - [Part passoires (année précédente)]
215
216
217 // --- Couleurs dynamiques -----
218
219 // Met en évidence l'année choisie dans les histogrammes et mini-courbes
220 // (fx → Valeur du champ, sur un visuel dont l'axe porte dim_date)
221 Couleur année choisie =
222 IF ( MAX ( dim_date[annee] ) = [Année choisie], "#2A78D6", "#B7D3F6" )
223
224 // =====
225 // ACCESSIBILITÉ ET TABLEAU DES COMMUNES
226 // =====
227
228 // Trois tranches pour colorer le nuage de points
229 // (Marqueurs → Couleur → fx → Valeur du champ)
230 Couleur accessibilité =
231 VAR a = [Années de niveau de vie pour acheter]
232 RETURN
233 SWITCH (
234     TRUE (),
235     ISBLANK ( a ), BLANK (),
236     a < 5, "#86B6EF",
237     a < 8, "#2A78D6",

```

```

238 |         "#104281"
239 |     )
240 |
241 | // Médiane sur les communes (pour les cartes de la page Accessibilité)
242 | Années pour acheter · commune médiane =
243 | MEDIANX (
244 |     FILTER ( VALUES ( dim_commune[code_commune] ), [Nb ventes] >= 10 ),
245 |     [Années de niveau de vie pour acheter]
246 | )
247 |
248 | Communes à 8 ans ou plus =
249 | COUNTRROWS (
250 |     FILTER (
251 |         VALUES ( dim_commune[code_commune] ),
252 |         [Nb ventes] >= 10 && [Années de niveau de vie pour acheter] >= 8
253 |     )
254 | )
255 |
256 | Niveau de vie · commune médiane =
257 | MEDIANX ( VALUES ( dim_commune[code_commune] ), [Niveau de vie médian] )
258 |
259 | // Évolution entre deux périodes fixes, pour le tableau des communes
260 | Prix m² médian 2021-2022 =
261 | CALCULATE ( [Prix m² médian], dim_date[annee] IN { 2021, 2022 } )
262 |
263 | Prix m² médian 2024-2025 =
264 | CALCULATE ( [Prix m² médian], dim_date[annee] IN { 2024, 2025 } )
265 |
266 | Évol. prix m² 2021-22 → 2024-25 =
267 | DIVIDE ( [Prix m² médian 2024-2025] - [Prix m² médian 2021-2022], [Prix m² médian 2021-2022] )

```

Les fonctions utilisées, expliquées

Fonction	Ce qu'elle fait	Exemple dans le fichier
COUNTRROWS	Compte les lignes de la table dans le contexte actuel	Nb ventes
MEDIAN	Médiane d'une colonne, recalculée pour les ventes filtrées	Prix m² médian
CALCULATE	Évalue une expression en modifiant le contexte de filtre. C'est la fonction centrale de DAX.	Toutes les mesures N-1
SAMEPERIODLASTYEAR	Décale la période filtrée d'un an en arrière (exige une table de dates marquée)	Nb ventes N-1
DATESINPERIOD	Fenêtre de dates glissante à partir d'une date	12 mois glissants
TOTALYTD	Cumul depuis le 1er janvier	Nb ventes cumul annuel
DIVIDE	Division qui renvoie vide au lieu d'une erreur si le dénominateur vaut 0	Évolutions, parts
SELECTEDVALUE	La valeur d'une colonne si une seule est filtrée, sinon vide (ou une valeur par défaut)	Niveau de vie médian
MEDIANX / FILTER / VALUES	Itère sur une liste (ici les communes) et calcule la médiane d'une expression	Commune médiane
TREATAS	Applique une liste de valeurs comme filtre sur une colonne, sans relation dans le modèle	Variante « année choisie »
FORMAT	Transforme un nombre en texte formaté. Le séparateur décimal s'écrit toujours « . » dans le format, Power BI affiche la virgule française.	Libellés avec flèches
SWITCH(TRUE(), ...)	Enchaîne des conditions, comme un CASE WHEN	Couleur accessibilité

Décision prise : le segment Année n'existe que sur la page 1

Sur les autres pages, les graphiques montrent toute la période. Un segment normal sur `dim_date[annee]` suffit donc, avec les mesures N-1 basées sur `SAMEPERIODLASTYEAR`. La carte et les mini-courbes de la page 1 ignorent le segment grâce à « Modifier les interactions » → « Aucun ». La table déconnectée `Années` reste dans le fichier comme variante optionnelle, si un jour un graphique doit garder toutes les années tout en mettant l'une d'elles en évidence.

Une fois le rapport enregistré en `.pbip`, les mesures sont versionnées dans le modèle (fichier `_Mesures.tmdl`) : `mesures.dax` pourra alors être supprimé pour garder une seule source de vérité.

7.4 Le rapport visé

Deux maquettes HTML ont été publiées pour montrer le rendu attendu, avec les vrais chiffres de la région. La deuxième, « version réalisable », ne contient que des visuels natifs de Power BI Desktop, avec la recette de chacun sous le graphique.

Page	Contenu	Points techniques
1. Vue d'ensemble	4 cartes de chiffres clés, carte des prix au m ² par commune, ventes par mois	Segment Année local, mesures N-1, interactions désactivées sur la carte
2. Marché immobilier	Prix médian sur 12 mois glissants, ventes par année, tableau des communes	DATESINPERIOD, étiquettes personnalisées, signets pour les tris
3. Accessibilité	Nuage prix / niveau de vie, communes les moins accessibles	Couleur par tranche via fx, MEDIANX sur les communes
4. Performance énergétique	Étiquettes A→G par période de construction, part de passoires dans le temps	Couleurs officielles du DPE, ligne de référence au 1er janvier 2025

Prochaine étape prévue : une mesure DAX qui dessine l'étiquette DPE officielle en SVG (flèches colorées de A à G et pointeur sur l'étiquette la plus fréquente), affichée comme image dans un tableau ou une carte.

8. Aide-mémoire

8.1 Relancer tout le pipeline

Terminal (Mac)

```
cd ~/Documents/open-data-logement
source .venv/bin/activate

# 1. Télécharger (idempotent : ne refait que ce qui manque)
python ingestion/download_dvf.py
python ingestion/download_insee.py
caffeinate -i python ingestion/download_dpe.py

# 2. Charger dans DuckDB
python ingestion/load_raw.py

# 3. Transformer et tester
cd dbt && dbt build --profiles-dir . && cd ..

# 4. Exporter vers Neon
python export/export_neon.py

# 5. Versionner
git add . && git commit -m "Mise à jour des données" && git push

# Puis dans Power BI (Parallels) : Accueil → Actualiser
```

Mise à jour	Fréquence	Commande
DVF	Deux fois par an (avril, octobre)	<code>download_dvf.py --force</code> puis la suite
DPE	En continu	<code>download_dpe.py --force</code>
INSEE	Une fois par an	<code>download_insee.py --force</code>

8.2 Glossaire

Terme	Définition
ELT / ETL	Extract-Load-Transform : charger brut puis transformer dans l'entrepôt. ETL transforme avant de charger.
Grain	Ce que représente une ligne d'une table (une vente, un logement, une commune × un mois...).
Idempotent	Qu'on peut relancer autant de fois qu'on veut avec le même résultat, sans doublon.
DuckDB	Base de données analytique sans serveur, stockée dans un fichier.
Parquet / NDJSON	Parquet : format colonnaire typé et compressé. NDJSON : un objet JSON par ligne.
Pagination	Découpage d'un résultat d'API en pages successives, reliées par un lien « next ».
Modèle dbt	Fichier SELECT que dbt transforme en vue ou en table.
<code>ref()</code> / <code>source()</code>	Références dbt vers un autre modèle ou une source ; elles construisent le graphe de dépendances.
Seed	Petit CSV versionné chargé comme table par dbt.
Macro / Jinja	Fonction de gabarit qui génère du SQL.

Terme	Définition
Test générique / singulier	Générique : réutilisable et déclaré en YAML (not_null...). Singulier : requête SQL écrite à la main qui doit renvoyer zéro ligne.
Schéma en étoile	Tables de faits (événements mesurés) reliées à des dimensions (axes d'analyse).
Mesure additive	Qu'on peut sommer sur n'importe quel axe (un comptage). Une médiane ne l'est pas.
Secret statistique	Règle qui interdit à l'INSEE de publier une valeur calculée sur trop peu de personnes.
Contexte de filtre	Ensemble des filtres actifs (segments, axes, lignes) sous lequel une mesure DAX est évaluée.
Table déconnectée	Table sans relation, lue par des mesures (TREATAS) pour piloter un calcul sans filtrer le modèle.
VEFA	Vente en l'état futur d'achèvement : vente sur plan, exclue des prix du marché de l'existant.
BAN	Base Adresse Nationale : référentiel des adresses, utilisé par l'ADEME pour géocoder les DPE.